ADVERSARIAL DETECTION IN INTRUSION DETECTION SYSTEMS USING ADVANCED MACHINE LEARNING TECHNIQUES

A PROJECT REPORT

Submitted by

HEMAVATHI R (422421106010)

JAEL BENEDICTIA J (422421106011)

KAJOEL P (422421106013)

in partial fulfillment for the award of the degree

Of

BACHELOR OF ENGINEERING IN

ELECTRONICS AND COMMUNICATION ENGINEERING

UNIVERSITY COLLEGE OF ENGINEERING, TINDIVANAM

ANNAUNIVERSITY:CHENNAI-600025

MAY-2025

ANNA UNIVERSITY: CHENNAI - 600 025

BONAFIDE CERTIFICATE

Certified that this project report **"ADVERSARIAL DETECTION IN INTRUSION DETECTION SYSTEMS USING ADVANCED MACHINE LEARNING TECHNIQUES"** is the bonafide work of **"R.HEMAVATHI (422421106010), J.JAEL BENEDICTIA (422421106011), P.KAJOEL (422421106013)"** who carried out the project work under my supervision.

SIGNATURE
Dr. K. VenkataLakshmi M.E., Ph.D.,
HEAD OF THE DEPARTMENT
Assistant Professor (SGL),
Department of ECE,
University College of Engineering,
Tindivanam,
Melpakkam - 604001.

SIGNATURE
Dr. S.S. Darly, M.E., Ph.D.,
SUPERVISOR
Assistant Professor (SGL),
Department of ECE,
University College of Engineering,
Tindivanam,
Melpakkam - 604001.

Submitted for University Project Viva-Voce held on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

At the outset we like to express our gratitude to the God almighty. We express our thanks to our **Dean (I/C) Dr. P. Thamizhazhagan, B.E., M.E., Ph.D.,** for his constant inspiration and encouragement throughout the Technical aspects of our project.

We would like to express our heartfelt gratitude and extend our sincere thanks to our Head of the Department, **Dr.K.Venkatalakshmi, M.E., Ph.D.** We are also thankful to **Dr.S.S.Darly, M.E., Ph.D.,** our internal project guide, for her valuable guidance and constant encouragement throughout the successful completion of our project work. We would further like to extend our gratitude to **Mr.D.Kadhiravan, B.Tech., M.Tech., Dr.P.Gajalakshmi, M.E.,Ph.D.,** for his consistent support, expert guidance, and thoughtful suggestions, which played a significant role in shaping our project.

We thank all our teaching and non-teaching faculty members of the department and also our fellow friends for helping us in providing valuable suggestions and timely ideas for the successful completion of the technical aspects of our project.

And also,we extend our immense thanks to our parents and siblings who have been a great source of inspiration and rendered us great support during the course of this project work. We sincerely thank all of them.

Table of Contents

1.1 Background 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scope of the Project 2

1.5 Methodology Overview 3

1.6 Organization of the Report 3

2. LITERATURE REVIEW 4

2.1 Evolution of Intrusion Detection Systems and the Shift Towards Machine Learning 7

2.1.1. Intrusion Detection Systems and Traditional Approaches 8

2.1.2 Emergence of Machine Learning in IDS 9

2.1.3 Deep Learning Advancements 9

2.1.4 Adversarial Machine Learning in IDS 10

2.1.5 Defense Mechanisms Against Adversarial Attacks 10

2.1.6 Dataset Considerations for IDS and Adversarial Detection 11

2.1.7 Hybrid and Ensemble Models 11

2.1.8 Real-Time Implementation Challenges 12

2.1.9 Interpretability and Explainability 12

2.1.10 Future Trends in Adversarial Detection for IDS 13

SYSTEM DESIGN 18

3.1 System Architecture Overview 18

3.2 Baseline CNN Training Using Preprocessed KDDTrain 20

3.3 Adversarial Testing of CNN Using FGSM –Generated Samples 22

3.4 GAN Training on KDD Attack Data 23

3.5 CNN Retraining with GAN-Generated Attack Samples 24

3.6 Final Model Testing and Robustness Evaluation 24

3.7 Modular Components 25

3.7.1 Data Preprocessing Module 25

3.7.2 CNN Classifier Module 25

3.7.3 FGSM Attack Generation Module 26

3.7.4 GAN Training Module 26

3.7.5 Augmented Training Module 27

3.7.6 Evaluation and Robustness Testing Module 27

IMPLEMENTATION 28

4.1 Dataset Preprocessing 28

4.2 Model Configurations 28

4.3 Algorithms Used 30

4.3.1 Convolutional Neural Network (CNN) 30

4.3.2 Fast Gradient Sign Method (FGSM) 33

4.3.3 Generative Adversarial Network (GAN) 34

4.4 Development Tools and Environment 38

RESULTS AND DISCUSSION 39

5.1 Baseline CNN Performance on Clean Data 39

5.3 GAN-Enhanced CNN Results 45

5.4 Confusion Matrix Analysis 49

5.5 ROC and AUC Analysis 50

5.6 Discussion and Insights 50

5.6.1 Vulnerability of Traditional CNN Models 50

5.6.2 Impact of FGSM Attack 50

5.6.3 Effectiveness of GAN-Augmented Adversarial Training 51

5.6.4 Model Generalization and Real-World Readiness 51

5.6.5 Strategic Implications 51

5.6.6 Limitations and Future Considerations 51

CONCLUSION 57

6.1 Future Work 58

6.1.1 Advanced Adversarial Attack Methods 58

6.2.2 Adaptive GAN Architectures 58

6.2.3 Ensemble and Hybrid Models 58

6.2.4 Real-Time Intrusion Detection 59

6.2.5 Transfer Learning and Cross-Dataset Testing 59

6.2.6 Explainability and Trust 59

6.3 Final Thoughts 60

REFERENCES 61

List Of Tables

Table 1 : Cnn Performance Of Clean Data 39

Table 2: Performance Against Fgsm Attack 44

Table 3 : Gan-Enhanced Cnn Results 46

Table 4: Confusion Matrix Analysis 49

List Of Figures

Figure 1: Cyber Security 1

Figure 2 : System Architecture 19

Figure 3: Method That Adversarial Attack Occur In System Model ..... **Error! Bookmark not defined.**

Figure 4 : Architecture Of CNN 32

Figure 5 : Architecture Of GAN 36

Figure 6: Confusion Metrics Of CNN Before FGSM 40

Figure 7 : Bar Graph For Accuracy 42

Figure 8 : Bar Graph For Precision 43

Figure 9 : Bar Graph For Recall 43

Figure 10 : Bar Graph For F1-Score 44

Figure 11 : Confusion Metrics Of Cnn After Fgsm 45

Figure 12: Confusion Metrics Of Gan Before Fgsm 46

Figure 13 : Confusion Metrics Of Gan After Fgsm 49

List Of Abbrevations

ABBREVIATION	FULL FORM
IDS	Intrusion Detection System
CNN	Convolutional Neural Network
FGSM	Fast Gradient Sign Method
GAN	Generative Adversarial Network
DL	Deep Learning
ML	Machine Learning
NSL-KDD	Network Security Lab - Knowledge Discovery in Databases
DOS	Denial of Service
R2L	Remote to Local
U2R	User to Root
PGD	Projected Gradient Descent
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
SVM	Support Vector Machine
KNN	K-Nearest Neighbors
SGD	Stochastic Gradient Descent
LSTM	Long Short-Term Memory
RNN	Recurrent Neural Network
ReLU	Rectified Linear Unit
API	Rectified Linear Unit

ABSTRACT

Cybersecurity has become increasingly vital with the rapid digitization of services and infrastructure. Intrusion Detection Systems (IDS) play a key role in detecting unauthorized or malicious activities within networks. Traditional IDS systems—either signature-based or anomaly-based—have given way to more intelligent, machine learning-powered systems due to their ability to identify patterns and behaviors that deviate from the norm. However, machine learning models, particularly deep learning architectures like Convolutional Neural Networks (CNNs), face critical vulnerabilities in the form of adversarial attacks. These attacks involve subtle, targeted modifications to input data that can mislead models into misclassifying malicious data as benign. A common adversarial method is the Fast Gradient Sign Method (FGSM), which uses gradient-based perturbations to create deceptive samples. These weaknesses pose serious risks to IDSs deployed in real-world environments, where attackers can easily exploit them. This project proposes an adversarially robust IDS by integrating CNNs with Generative Adversarial Networks (GANs). The goal is to improve resilience against adversarial inputs by augmenting the training data with GAN-generated attack samples.

CHAPTER- 1

INTRODUCTION

1.1 Background

In the current digital era, the expansion of internet-connected systems and the digitization of information have made cybersecurity a critical concern. Organizations across all sectors, from finance and healthcare to government and defense, depend heavily on network infrastructure to support operations, manage services, and store sensitive data. As a result, cyber threats have evolved in sophistication, scale, and frequency. Intrusion Detection Systems (IDS) have become indispensable tools for monitoring and defending against such threats by identifying unauthorized or anomalous activities within networks.

Traditional IDS models rely heavily on signature-based detection methods, which use predefined rules to flag known attacks. While effective against previously seen threats, these systems often fail to detect novel or zero-day attacks. To overcome these limitations, researchers have turned to intelligent IDS models built using machine learning (ML) and deep learning (DL) techniques. These models can learn and adapt to complex patterns in network traffic, enabling them to detect both known and unknown threats.

Figure 1: cyber security

1.2 Problem Statement

Despite the success of ML and DL techniques in IDS, they are not immune to challenges. A growing concern is their vulnerability to **adversarial attacks**—a class of attacks that subtly manipulate input data to mislead the learning model. These perturbations are often imperceptible to humans but can cause drastic misclassification in trained neural networks. Such vulnerabilities pose a major security threat, especially in critical systems relying on automated detection.

This research addresses the challenge of adversarial vulnerability in deep learning-based IDS, particularly focusing on attacks generated using the **Fast Gradient Sign Method (FGSM).** The goal is to improve IDS robustness by integrating Generative **Adversarial Networks (GANs)** into the training process to simulate attack behaviors and strengthen detection capabilities.

1.3 Objectives

The main objectives of this project are:

- To investigate the susceptibility of CNN-based IDS to adversarial attacks.
- To implement FGSM to generate adversarial attack samples.
- To design and train a GAN model capable of producing realistic attack data.
- To retrain the IDS using a combination of clean and GAN-generated samples.
- To evaluate and compare the robustness of the baseline and improved IDS models.

1.4 Scope of the Project

This project focuses on improving the adversarial robustness of a CNN-based IDS using synthetic attack data generated by a GAN. The dataset used is the **NSL-KDD,** a widely accepted benchmark for evaluating IDS. The scope is limited to evaluating the performance of the proposed model under white-box attacks, specifically using FGSM. Real-time deployment and defense against more advanced attacks like PGD or black-box scenarios are beyond the current project's scope but provide future directions.

1.5 Methodology Overview

The system is built in multiple stages. Initially, a baseline CNN is trained on clean NSL-KDD data. Then, FGSM is applied to generate adversarial samples, and the CNN's performance is evaluated against these inputs. A GAN is trained on malicious traffic data to produce synthetic attacks, which are then used to augment the dataset. Finally, the CNN is retrained using both clean and synthetic data to form a more robust model. Performance is measured using metrics like accuracy, precision, recall, F1-score, and confusion matrices.

1.6 Organization of the Report

This report is organized into the following chapters:

- **Chapter 1: Introduction –** Provides background, problem definition, objectives, and project scope.
- **Chapter 2: Literature Review –** Discusses prior work related to IDS, adversarial attacks, CNNs, GANs, and defense mechanisms.
- **Chapter 3: System Design –** Describes the architecture, data flow, and modular components of the proposed system.
- **Chapter 4: Implementation –** Details dataset preprocessing, model configurations, algorithms, and development tools.
- **Chapter 5: Results and Discussion –** Analyzes experimental outcomes and compares model performances.
- **Chapter 6: Conclusion and Future Work –** Summarizes findings and outlines potential future extensions.

CHAPTER 2

LITERATURE REVIEW

2. LITERATURE REVIEW

Intrusion Detection Systems (IDS) are a fundamental part of cybersecurity architecture, designed to detect unauthorized access, malicious behavior, or policy violations in computer networks. Over time, IDS technologies have evolved from simple rule-based systems to complex, adaptive intelligence-driven frameworks.

Early IDS models, such as the one proposed by Dorothy Denning in 1987, laid the foundation for automated security monitoring. Signature-based detection systems, like Snort, matched network activity with a known set of attack signatures. These systems performed well against known threats but failed to detect new or mutated attacks due to their reactive nature.

To overcome this, anomaly-based detection methods were introduced. These systems learned patterns of "normal" behavior and flagged deviations as potential intrusions. While more proactive, these methods suffered from high false-positive rates. Lunt et al. (1992) used statistical anomaly detection but encountered challenges in scalability and adaptability.

Machine Learning (ML) techniques marked a significant evolution in IDS. They enabled models to learn from past data and identify malicious patterns without relying on fixed rules. Early works like those of Lee and Stolfo (2000) applied data mining techniques to automate intrusion detection, making systems more intelligent and adaptable.

Various ML algorithms were adopted, including Support Vector Machines (SVM), Decision Trees, Random Forests, and K-Nearest Neighbors (KNN). These models improved detection accuracy but still struggled with unseen attack types and required significant feature engineering.

As cyber threats became more complex, researchers turned to Deep Learning (DL), which allowed models to learn deeper data representations. CNNs, RNNs, LSTMs, and Autoencoders became popular in IDS research. CNNs were used to detect spatial patterns in network data, while RNNs and LSTMs captured temporal sequences.

Yin et al. (2017) demonstrated the use of DL on the NSL-KDD dataset, achieving higher accuracy and robustness compared to traditional ML approaches. DL models reduced the need for manual feature selection but were resource-intensive and operated as black boxes, limiting interpretability.

One of the critical vulnerabilities of DL models is their susceptibility to adversarial attacks. Adversarial Machine Learning (AML) is a field that studies how carefully crafted inputs can fool ML/DL models. These inputs often look normal but cause misclassification.

Biggio et al. (2013) were among the first to explore adversarial attacks in security systems. Goodfellow et al. (2015) introduced the Fast Gradient Sign Method (FGSM), which subtly perturbs inputs in the direction of the model's loss function to cause incorrect predictions.

FGSM showed that even state-of-the-art DL models in IDS could be fooled with minimal changes to input data. This revelation prompted a surge in research focused on defending against such attacks.

Defense mechanisms such as adversarial training, where models are trained with both clean and adversarial examples, were proposed. Though effective, adversarial training is computationally expensive. Other defenses include input transformation, feature squeezing, and gradient masking.

Zhang et al. (2019) proposed a hybrid defense combining adversarial training and feature selection to enhance interpretability and robustness. Guo et al. (2018) introduced input transformation techniques, while Xie et al. (2019) used feature denoising for adversarial defense.

Datasets play a vital role in IDS research. The NSL-KDD dataset, an improved version of KDD'99, is commonly used but lacks modern attack types. CICIDS2017 and UNSW-NB15 are newer, more realistic datasets that include a wider variety of attack vectors.

However, these datasets do not naturally include adversarial samples. Researchers generate synthetic adversarial data using techniques like FGSM, DeepFool, Carlini & Wagner (C&W), and GANs to evaluate model robustness under attack.

Generative Adversarial Networks (GANs), introduced by Goodfellow et al. in 2014, have become crucial for adversarial sample generation. GANs consist of a generator and a discriminator that work in opposition to produce realistic data.

In IDS research, GANs are used to create attack-like inputs that mimic real malicious traffic. These samples are then used to retrain IDS models like CNNs to improve their robustness against adversarial attacks.

Sharif et al. (2016) demonstrated the real-world impact of adversarial examples in facial recognition systems, proving the effectiveness of physical adversarial inputs. These ideas have transferred to network security research.

To enhance performance, hybrid models that combine CNNs with LSTMs or autoencoders are used. These models exploit both spatial and temporal patterns in network data, capturing complex behaviors more effectively.

Ensemble methods like bagging, boosting, and stacking also improve detection accuracy and robustness. Goyal et al. (2018) used an ensemble of neural networks and Random Forests to achieve better generalization on NSL-KDD.

Interpretability has become a major focus in IDS development. Tools like LIME (Ribeiro et al., 2016) and SHAP help explain why a model flagged an input as malicious, improving trust in automated detection systems.

Real-time implementation of DL-based IDS faces challenges such as high computational requirements and latency. Techniques like model compression and pruning help reduce resource consumption without significant loss of accuracy.

Federated learning is emerging as a privacy-preserving approach where models are trained across decentralized systems without sharing raw data. This is particularly useful in distributed IDS systems across multiple organizations.

Blockchain has also been proposed for secure logging and decentralized trust in IDS. Combined with ML, it offers a tamper-proof method of sharing threat intelligence across networks.

Neuromorphic computing, which mimics human brain structures, is another futuristic approach being explored for low-power, real-time IDS applications.

As IDS becomes more intelligent, it also becomes a target. Adversarial attacks like evasion, poisoning, and backdoor attacks must be considered in model development and deployment.

A comprehensive literature base shows that while DL improves detection capabilities, it also opens the door to new security challenges. Therefore, developing IDS systems that are both accurate and adversarially robust is critical.

Hybrid models, robust datasets, adversarial training, and explainability tools must be integrated into future IDS research to build truly secure systems.

This literature survey establishes the foundation for our project, which aims to enhance IDS performance using CNN and GAN-based adversarial detection techniques. By identifying and defending against adversarial inputs, we contribute to building smarter, more resilient cybersecurity solutions.

2.1 Evolution of Intrusion Detection Systems and the Shift Towards Machine Learning

The increasing complexity and volume of cyber threats over the past few decades have made network security a vital area of research and implementation. Intrusion Detection Systems (IDS) are at the forefront of this defense strategy, tasked with monitoring network traffic for signs of malicious activity and alerting administrators to potential threats. Traditionally, IDS relied heavily on signature-based detection, a method that compares network activity to a predefined database of known attack signatures. While this approach is efficient for identifying previously encountered threats, it suffers from a major drawback—it cannot detect zero-day attacks or novel variations of existing exploits, since no signature exists for these.

To overcome this limitation, researchers introduced anomaly-based detection, which involves identifying deviations from established normal behavior patterns in network activity. However, early statistical and rule-based anomaly detectors often resulted in high false-positive rates and lacked the ability to adapt to new types of attacks. This inefficiency led to a growing interest in leveraging machine learning (ML) for IDS, as ML models can learn from data and automatically identify complex patterns without the need for manually crafted rules.

As early as the late 1990s and early 2000s, studies like those by Lee and Stolfo (2000) began exploring data mining techniques for IDS, laying the groundwork for more intelligent and adaptive systems. ML-based IDSs offered the potential to generalize beyond fixed rules, detect subtle anomalies, and adapt to new attack behaviors. However, while promising, these models required high-quality labeled data and often struggled with high-dimensional network data, necessitating careful feature engineering and preprocessing.

The last decade has seen a sharp increase in the adoption of deep learning (DL) techniques—especially Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Autoencoders—to further enhance IDS capabilities. These models excel at feature extraction and can handle large, complex datasets with minimal manual intervention. Despite their success, DL-based IDS have introduced a new vulnerability: adversarial attacks. These attacks subtly manipulate input data in a way that causes the model to misclassify malicious inputs as benign, often without detection. This challenge has sparked an entirely new field—adversarial machine learning (AML)—which now plays a critical role in the development of secure and robust IDS frameworks.

2.1.1. Intrusion Detection Systems and Traditional Approaches

Early research in Intrusion Detection Systems (IDS) began with Dorothy Denning's foundational model in 1987, which introduced the concept of automated security monitoring. These systems were designed to detect unauthorized access or abnormal activity within a computer network. A major advancement came with the introduction of signature-based IDS, such as Snort. These systems use predefined patterns or "signatures" of known threats to detect malicious activity. While effective against known threats, they are inherently reactive and unable to detect novel or zero-day attacks or polymorphic malware, which can morph their characteristics to evade detection.

Anomaly-based IDS attempted to overcome these limitations by learning what constitutes "normal" behavior and flagging deviations from this baseline. This method is more proactive, capable of identifying unknown attacks. However, a major downside is its high false-positive rate, where benign behavior is misclassified as malicious. Early works like those of Lunt et al. in 1992 leveraged statistical anomaly detection methods, which brought new perspectives but struggled with adaptability and scalability, limiting their

effectiveness in evolving and large-scale networks. As cyber threats became increasingly sophisticated, these traditional approaches proved inadequate, paving the way for data-driven methods.

2.1.2 Emergence of Machine Learning in IDS

Machine Learning (ML) techniques brought a paradigm shift to IDS by enabling systems to learn from past data and make data-driven decisions. Instead of relying solely on static rules or statistical thresholds, ML algorithms like decision trees, support vector machines (SVM), and k-nearest neighbors (KNN) began to be applied for detecting patterns and anomalies in network traffic.

Lee and Stolfo's work in 2000 was instrumental in this transformation. They introduced a data mining framework for intrusion detection, enabling systems to automatically learn discriminative features from network traffic. Their approach laid the groundwork for further research into ML-based IDS. Over time, researchers explored supervised, unsupervised, and semi-supervised learning to improve detection capabilities.

Despite their promise, traditional ML models required extensive feature engineering and were often susceptible to overfitting or misclassification when faced with adversarial inputs or previously unseen attack types. These limitations created a need for more advanced models capable of learning deeper, more abstract representations of data.

2.1.3 Deep Learning Advancements

Deep Learning (DL) has significantly advanced IDS by providing models capable of automatically extracting high-level features from raw data. Techniques like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Autoencoders are now widely used in IDS research. CNNs can capture spatial patterns in structured data, RNNs are effective in modeling sequential patterns, and Autoencoders help detect anomalies by reconstructing input data.

Yin et al. (2017) applied DL models to the NSL-KDD dataset, demonstrating superior detection accuracy and speed compared to traditional ML methods. DL's ability to model

complex relationships without extensive manual feature selection made it appealing for intrusion detection. However, DL also introduced new challenges: models are computationally intensive, often act as "black boxes," and are vulnerable to adversarial attacks. Despite these issues, DL remains a leading approach due to its ability to learn from large-scale, high-dimensional data.

2.1.4 Adversarial Machine Learning in IDS

Adversarial Machine Learning (AML) explores how malicious actors can manipulate machine learning models. Adversaries can introduce subtle perturbations to input data that lead to incorrect model predictions. Biggio et al. (2013) were pioneers in identifying this threat in the context of security systems, showing that ML models could be deceived by inputs crafted to exploit learned patterns.

Goodfellow et al. (2015) further advanced this field by introducing the Fast Gradient Sign Method (FGSM), an efficient technique for generating adversarial examples. FGSM computes the gradient of the model's loss with respect to the input and modifies the input slightly in that direction, causing the model to misclassify. Huang et al. (2011) had also earlier warned about the fragility of ML models in adversarial settings.

These findings alarmed the cybersecurity community, revealing that even state-of-the-art DL models could be bypassed. Consequently, adversarial robustness became a critical research objective for IDS.

2.1.5 Defense Mechanisms Against Adversarial Attacks

To combat adversarial threats, several defense mechanisms have been proposed. Among the most effective is **adversarial training**, where models are trained on both clean and adversarial examples to enhance resilience. Though computationally expensive, this technique helps models learn more robust decision boundaries.

Other methods include **gradient masking**, which hides gradient information to make it harder for attackers to craft adversarial inputs, and **feature squeezing**, which reduces input

sensitivity by compressing data representations. **Input preprocessing**, such as normalization or denoising, is also used to mitigate adversarial perturbations.

Wang et al. (2020) demonstrated that adversarial training significantly improved IDS performance under attack. Zhang et al. (2019) introduced a hybrid method combining adversarial training with feature selection, showing gains in both robustness and interpretability. These developments represent an ongoing effort to integrate defensive strategies directly into the model training process.

2.1.6 Dataset Considerations for IDS and Adversarial Detection

Datasets are crucial for developing and evaluating IDS models. Commonly used datasets include **NSL-KDD**, **CICIDS2017**, and **UNSW-NB15**. Each has its strengths and limitations. NSL-KDD, a cleaned-up version of the older KDD'99, is widely used but lacks diversity and real-time adversarial samples. CICIDS2017 offers more modern attack scenarios but is large and computationally demanding.

A major issue with these datasets is the lack of realistic adversarial examples. Researchers have resorted to synthetic generation of adversarial data using algorithms like FGSM, DeepFool, and GANs. Petras et al. (2021) stressed the need for continuous updates to datasets to reflect current threat landscapes, emphasizing the inclusion of adversarial and zero-day attack samples for better training and evaluation.

2.1.7 Hybrid and Ensemble Models

To increase the robustness and generalization of IDS, researchers have developed hybrid and ensemble models. Hybrid models combine the strengths of different DL architectures—e.g., CNNs for spatial features and LSTMs for temporal patterns—into a single framework. This allows the IDS to learn both static and dynamic characteristics of network traffic.

Ensemble methods such as bagging, boosting, and stacking aggregate the predictions of multiple models to improve overall performance and robustness. Kuppa and Rama Krishna (2020) introduced an ensemble IDS framework combining random forests, SVMs, and

neural networks. Their system showed enhanced resistance to adversarial inputs due to its diversified decision boundaries.

These techniques not only increase detection accuracy but also reduce the likelihood of a single point of failure, making the system more resilient to adversarial evasion techniques.

2.1.8 Real-Time Implementation Challenges

While many IDS models perform well in offline evaluations, deploying them in real-time environments presents several challenges. Deep learning models often have high **computational complexity**, leading to latency issues in processing live network traffic. Real-time systems demand low response times, which can be difficult to achieve with resource-intensive models.

To address these issues, researchers have explored techniques like **model compression**, which reduces the size and complexity of DL models without significantly compromising accuracy. **Transfer learning** is also used to reduce training time and allow models to adapt to new environments with minimal data. Despite these advancements, real-time deployment remains an ongoing challenge due to the trade-off between model complexity and responsiveness.

2.1.9 Interpretability and Explainability

As IDS systems increasingly rely on complex ML and DL models, their **lack of interpretability** has become a concern. Security analysts need to understand why a particular alert was triggered. Tools like **LIME (Local Interpretable Model-Agnostic Explanations)** and **SHAP (SHapley Additive exPlanations)** help visualize and explain model predictions.

Ribeiro et al. (2016) introduced LIME to provide local explanations by approximating complex models with interpretable ones around a specific prediction. These methods have since been adapted for cybersecurity, enabling analysts to trace the rationale behind IDS decisions and improve trust in automated systems.

2.1.10 Future Trends in Adversarial Detection for IDS

Looking ahead, emerging technologies are shaping the future of IDS. **Federated learning** is gaining traction as a privacy-preserving method for distributed model training without sharing raw data. This is particularly useful for collaborative IDS across multiple organizations. **Blockchain technology** is also being explored to ensure data integrity and decentralized security.

Neuromorphic computing, which mimics the brain's architecture, promises ultra-low-power IDS solutions capable of operating in edge environments. Self-healing and **adaptive IDS** are being developed to detect and recover from attacks dynamically, using reinforcement learning and continual learning methods.

These future trends highlight the ongoing evolution of IDS towards intelligent, adaptive, and decentralized systems that can withstand sophisticated adversarial threats in real-world scenarios.

Early research in IDS, such as that by Denning (1987), laid the foundational architecture for automated security monitoring. Signature-based IDS like Snort provided strong protection against known threats but were static and ineffective against zero-day attacks or polymorphic malware. Anomaly-based IDS sought to overcome this limitation by identifying deviations from normal behavior, but they often suffered from high false positive rates.

The work of Lunt et al. (1992) on anomaly detection highlighted the potential of statistical methods, but limitations in adaptability and scalability prompted researchers to explore data-driven solutions. As cyberattacks became more complex, it was clear that traditional methods could not keep pace.

2.2 Approches in Adversarial Datection for Intrusion Detection Systems (IDS)

1. Goodfellow et al. proposed the Fast Gradient Sign Method (FGSM), one of the first techniques to generate adversarial examples by adding small, imperceptible noise to input

data. This method exposed significant vulnerabilities in deep neural networks and laid the foundation for adversarial testing in security systems. Their work highlighted the importance of adversarial robustness and inspired further research into defense mechanisms.

2. Papernot et al. investigated the limitations of deep learning models in adversarial settings and introduced the concept of defensive distillation as a defense method. Their work demonstrated how neural networks can be manipulated at test time and emphasized the need for adaptive learning techniques to resist adversarial perturbations in real-time applications such as IDS.

3. Carlini and Wagner developed advanced adversarial attacks (C&W attacks) that could bypass many existing defense methods. Their study evaluated the robustness of neural networks and proposed optimized attack strategies. The results showed how traditional IDS models can be deceived even by sophisticated, well-trained classifiers.

4. Moosavi-Dezfooli et al. introduced DeepFool, a powerful adversarial attack algorithm that generates minimal perturbations to fool classifiers. This method was evaluated across multiple neural network architectures and demonstrated the ease with which even robust models could be fooled, thereby stressing the need for adversarially trained IDS systems.

5. Yuan et al. conducted a comprehensive survey on adversarial attacks and defenses in deep learning. The paper provided an overview of attacks such as FGSM, JSMA, and C&W, and discussed mitigation techniques including adversarial training and gradient masking. This work is highly relevant for designing secure IDS systems in adversarial environments.

6. Goodfellow et al. also introduced Generative Adversarial Networks (GANs), which are widely used in adversarial sample generation. GANs learn to produce data indistinguishable from real inputs, making them ideal tools for generating realistic attack traffic in IDS training and evaluation.

7. Akhtar and Mian provided an extensive survey on adversarial threats in deep learning, particularly in computer vision and security domains. The paper explored the risks posed by adversarial examples and proposed the need for explainable, interpretable models for secure applications like intrusion detection.

8. Mirsky et al. introduced Kitsune, an ensemble of autoencoders designed for online network intrusion detection. The model adapts in real-time and uses unsupervised learning, demonstrating strong potential for deployment in adversarially rich environments where pre-labeled data is limited.

9. Tavallaee et al. analyzed the weaknesses of the KDD Cup 1999 dataset and proposed the NSL-KDD dataset as an improved benchmark for IDS. Their work facilitated standardized testing for adversarial robustness in IDS research by providing a better-balanced dataset.

10. Goyal et al. proposed an ensemble method combining Random Forest and Neural Networks for IDS. Their approach showed improved detection accuracy on the NSL-KDD dataset, but did not address adversarial robustness, highlighting the need for future work incorporating adversarial defense strategies.

11. Guo et al. proposed input transformation techniques (e.g., cropping, compression, and bit-depth reduction) as defenses against adversarial images. While focused on vision, their methods inspire similar pre-processing defenses in IDS systems to reduce vulnerability to adversarial input manipulation.

12. Zhang et al. analyzed the limits of adversarial training and proposed a “blind-spot” attack that targets weaknesses not seen during training. The study emphasized the risk of over-relying on adversarial training alone and called for more generalizable IDS defense strategies.

13. Xie et al. introduced a novel method of feature denoising to defend against adversarial examples. Their approach enhances adversarial robustness by cleaning intermediate network representations, which can be adapted to enhance detection performance in IDS systems under attack.

14. Shokri et al. explored membership inference attacks, a privacy risk where adversaries determine if specific data was used during model training. While focused on privacy, this work is relevant for IDS models deployed in sensitive environments, underscoring the need for secure learning mechanisms.

15. Goodfellow et al. presented the original concept of Generative Adversarial Networks (GANs), which are now widely used for adversarial sample generation. In IDS, GANs help

simulate realistic attack traffic to train more resilient models capable of detecting unseen threats.

16. Dong et al. enhanced attack strength and transferability using momentum-based adversarial attacks. Their work demonstrated that combining optimization techniques with FGSM can produce more dangerous samples, guiding the development of stronger IDS defenses.

17. Huang et al. discussed adversarial attacks in deep reinforcement learning and neural network policies. Though outside traditional IDS, their insights into policy manipulation are applicable in smart IoT environments where automated decision-making is vulnerable.

18. Abadi et al. introduced TensorFlow, a major deep learning framework used in adversarial research. In IDS systems, TensorFlow supports the development of scalable CNN, GAN, and FGSM implementations for adversarial detection.

19. Keras Documentation provides APIs for building neural networks. Many IDS systems use Keras for rapid experimentation with CNNs and GANs, making it essential for prototyping and evaluating adversarial training strategies.

20. Pan and Yang conducted a survey on transfer learning, which is useful in IDS when adapting trained models to new network environments or datasets with limited labels. It's also useful in building adversarially trained models that generalize well.

21. LeCun, Bengio, and Hinton outlined the foundations of deep learning, including CNNs, which are heavily used in modern IDS frameworks. Their work enables understanding of how deep models learn and how they can be attacked or defended.

22. Bengio et al. discussed deep architectures in AI and their training complexities. This helps justify why GANs and CNNs are effective in modeling complex intrusion patterns and adversarial variations.

23. Jordan and Mitchell provided insights into the future of machine learning, including its role in security, policy, and regulation — areas crucial to deploying adversarial-resilient IDS in real-world environments.

24. Simonyan and Zisserman proposed the VGG network, a deep CNN model. While originally for vision, its deep layered design is adapted in IDS research for feature extraction and classification of network traffic.

25. Mirsky et al. proposed Kitsune, an online anomaly-based IDS using an ensemble of autoencoders. It adapts in real time and is suitable for environments where adversarial attacks constantly evolve.

26. Xu et al. reviewed adversarial attacks and defenses across images, graphs, and text. Their cross-domain analysis supports the idea that IDS systems must be designed to handle attacks in structured and unstructured network data.

27. Zhang et al. introduced HIDE, a hierarchical IDS using statistical preprocessing and neural networks. This architecture lays groundwork for combining traditional anomaly detection with deep learning under adversarial settings.

28. Akhtar and Mian conducted a thorough review of adversarial threats in deep learning, focusing on vision but offering strategies that apply to IDS, such as adversarial input detection and adversarial training.

29. Sharif et al. demonstrated real-world adversarial attacks against facial recognition using physical objects. Though focused on vision, it shows how subtle input changes can bypass high-performing models — a key concern for IDS in real-time traffic monitoring.

30. Tavallaee et al. performed foundational work in revising the KDD Cup dataset into NSL-KDD, which removes redundancy and supports fairer evaluation of IDS models under adversarial testing.

CHAPTER 3

SYSTEM DESIGN

3.1 System Architecture Overview

The system architecture for adversarial detection in **intrusion detection systems (IDS)** is designed as a multi-stage deep learning pipeline that systematically processes network data, simulates attacks, enhances robustness, and evaluates performance. It integrates **Convolutional Neural Networks (CNN),** the **Fast Gradient Sign Method (FGSM),** and **Generative Adversarial Networks (GANs)** to train a resilient classifier capable of detecting both standard and adversarial intrusions. As illustrated in Figure 2: System Architecture, the pipeline initiates with data collection and flows through preprocessing, initial CNN training, adversarial sample generation, GAN training, retraining with augmented data, and final evaluation. The architecture is modular, allowing independent development and optimization of each stage while maintaining a cohesive data flow.

The process begins with the acquisition of network traffic data, specifically the **NSL-KDD dataset,** a refined version of the **KDD'99 dataset** that addresses issues such as redundancy and imbalance. This dataset contains labeled instances of normal activity and various attack types (DoS, Probe, R2L, U2R), which are essential for training and evaluating the intrusion detection model. After data collection, the dataset undergoes preprocessing to prepare it for model input. This includes feature normalization using min-max scaling, encoding of categorical features such as protocol_type, service, and flag, and filtering and separation of attack and normal data, followed by division into training and testing sets. This ensures that the **CNN** and **GAN** modules receive structured, clean data inputs for effective learning.

Next, a baseline **CNN** is trained using the cleaned dataset to classify traffic as normal or malicious based on learned patterns. This baseline performance establishes a reference point for evaluating improvements from subsequent adversarial training. Once the **CNN** is trained, adversarial samples are generated using **FGSM,** which introduces perturbations to input samples by adjusting feature values along the gradient of the loss function. This step exposes the model's vulnerability to adversarial manipulation.

To strengthen the model's resilience, a **GAN** is trained using the attack-only subset of the **NSL-KDD dataset**. The generator within the **GAN** produces synthetic attack samples, while the

discriminator learns to distinguish real attacks from fake ones. Over time, the generator becomes proficient at mimicking real attack behavior, thereby enriching the attack dataset with diverse, realistic samples. The CNN is then retrained using an augmented dataset that includes both the original clean data and the GAN-generated attacks. This retraining phase equips the model with more robust decision boundaries, improving its ability to generalize across real and adversarial examples and counter evasion tactics effectively.

Finally, the performance of the retrained **CNN** is evaluated using clean test samples, FGSM-perturbed adversarial samples, and GAN-generated synthetic attacks. Metrics such as accuracy, precision, recall, F1-score, confusion matrix, and **AUC** are used to assess the model's effectiveness. The final **CNN** is expected to demonstrate high detection accuracy, low false positive rates, and strong resistance to adversarial manipulation. Overall, this architecture offers a structured, scalable, and adaptable solution for intrusion detection. By combining adversarial training with synthetic data augmentation, it significantly outperforms traditional IDS models in both clean and adversarial environments. The modular design also facilitates adaptation to new datasets, evolving threats, and emerging detection techniques.

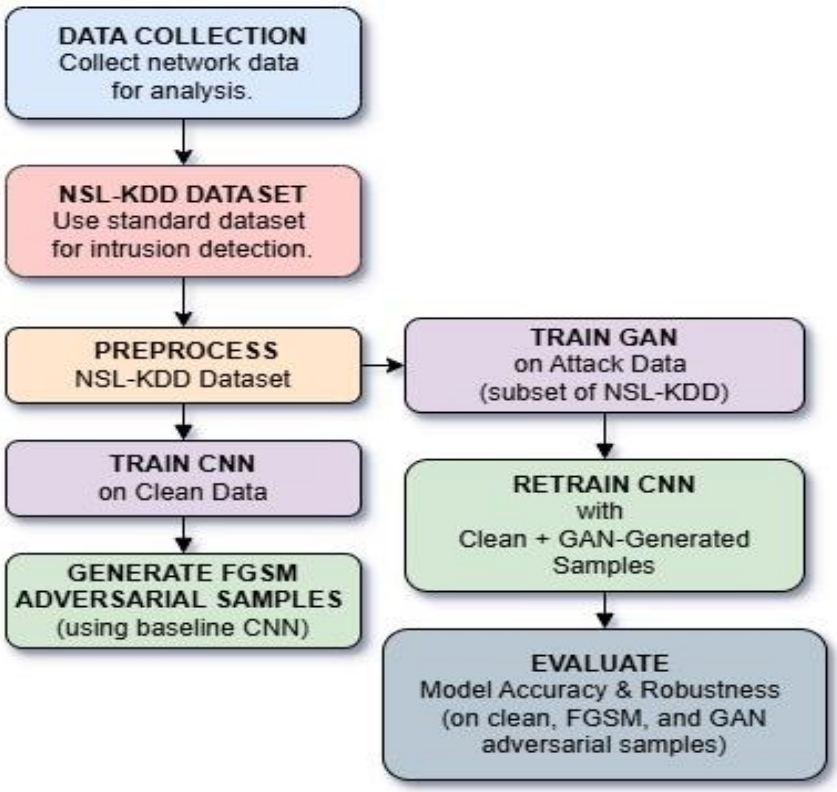


Figure 2 : System Architecture

This diagram depicts the adversarial training loop in GAN. The Generator learns to create fake attack samples from noise, while the Discriminator evaluates both real and fake inputs. Feedback from the Discriminator helps the Generator improve over time.

3.2 Baseline CNN Training Using Preprocessed KDDTrain .csv

The phase described marks the foundational step in constructing a robust adversarial-aware intrusion detection system. It begins with the **initial preprocessing** of the KDDTrain.csv dataset, a key component derived from the NSL-KDD dataset, widely used for evaluating network intrusion detection models. This dataset contains numerous instances of network traffic, labeled either as **normal** or belonging to specific **attack categories**, such as DoS (Denial of Service), Probe, R2L (Remote to Local), and U2R (User to Root). Each instance is characterized by multiple features, including both **numerical** (e.g., duration, source bytes, destination bytes) and **categorical** (e.g., protocol type, service, flag) variables.

The **preprocessing step** is essential for ensuring data quality and consistency before feeding it into a deep learning model. It starts with **data cleaning**, which involves removing or imputing **null or missing values** that could disrupt the model's learning process. While the NSL-KDD dataset is generally clean, additional verification is crucial for ensuring there are no corrupted entries or anomalies that could bias the model.

Following data cleaning, **feature normalization** is applied. This typically involves using **min-max scaling** or **z-score standardization** to rescale numerical attributes to a uniform range, often between 0 and 1. Normalization is critical because CNNs (Convolutional Neural Networks), like most neural networks, perform better when input features are on a similar scale, allowing for more stable and faster convergence during training.

Next, **categorical variables** such as protocol_type, service, and flag must be **encoded** into a numerical format since neural networks cannot directly interpret text or symbolic data. Common methods include **one-hot encoding** or **label encoding**. One-hot encoding is typically preferred in this context as it avoids imposing a false ordinal relationship between categories.

Another important preprocessing step is **data balancing**. In the original NSL-KDD dataset, some classes—particularly attack categories like U2R—are severely underrepresented. This imbalance can cause the model to become biased toward majority classes, leading to high overall

accuracy but poor detection of rare attack types. Techniques such as **oversampling**, **undersampling**, or **SMOTE (Synthetic Minority Over-sampling Technique)** may be employed to address this issue, ensuring that the model has enough examples of each class to learn effectively.

After preprocessing, the dataset is **divided into training and testing subsets**. This is crucial for evaluating model generalization. Typically, a split like 80:20 or 70:30 is used, ensuring that the testing set includes both normal and attack instances that the model has not seen during training. This separation prevents data leakage and allows for unbiased performance evaluation.

With the dataset prepared, a **baseline CNN model** is constructed. While CNNs are more commonly associated with image processing, they have also shown promise in time-series and structured tabular data like network traffic records. The CNN architecture typically begins with an **input layer** matching the dimensionality of the preprocessed feature vector. This is followed by one or more **convolutional layers** that apply filters to detect patterns or features in the data.

Pooling layers, such as **max pooling**, are then used to reduce dimensionality and retain the most prominent features, improving computational efficiency and reducing overfitting. The output from these layers is then passed through one or more **dense (fully connected) layers**, where the network learns higher-level feature representations. The final **output layer** typically uses a **softmax** or **sigmoid** activation function to output classification probabilities, indicating whether an input instance is normal or represents a specific type of attack.

Once the architecture is defined, the model is **trained** using the training dataset. During training, the CNN adjusts its internal weights using **backpropagation** and an **optimization algorithm** such as **Adam** or **SGD (Stochastic Gradient Descent)**. The model's objective is to minimize a **loss function**, often **binary cross-entropy** or **categorical cross-entropy**, depending on whether the classification is binary (normal vs. attack) or multi-class (normal + multiple attack types).

Throughout the training process, various **metrics**—including **accuracy**, **loss**, **precision**, **recall**, and **F1-score**—are monitored to evaluate learning progress and detect signs of overfitting or underfitting. Techniques like **early stopping**, **dropout**, and **batch normalization** may also be employed to further improve generalization.

After training is complete, the CNN is evaluated on the clean, unseen testing data. This provides a **baseline performance measure**, reflecting the model's ability to detect attacks under standard

conditions without any adversarial manipulation. This performance serves as a **reference point** for all subsequent evaluations, particularly when adversarial attacks and defense mechanisms are introduced in later phases.

Establishing this baseline is vital. It provides insight into the strengths and weaknesses of the model under normal circumstances and helps quantify the **impact of adversarial attacks** and the **effectiveness of countermeasures** implemented in later stages. Without this phase, it would be difficult to determine whether subsequent improvements or degradations in performance are due to changes in model architecture, training data, or adversarial influences.

3.3 Adversarial Testing of CNN Using FGSM –Generated Samples

Using the trained baseline CNN, FGSM (Fast Gradient Sign Method) is employed to generate adversarial examples. FGSM perturbs input features slightly in the direction of the gradient to trick the CNN into misclassifying malicious samples as benign. These adversarial inputs are tested against the baseline CNN, revealing vulnerabilities in the model. Typically, accuracy drops significantly, demonstrating the fragility of conventional CNNs to adversarial manipulation. This step emphasizes the need for stronger, adversarially-aware training techniques and forms a baseline for measuring the success of future improvements.

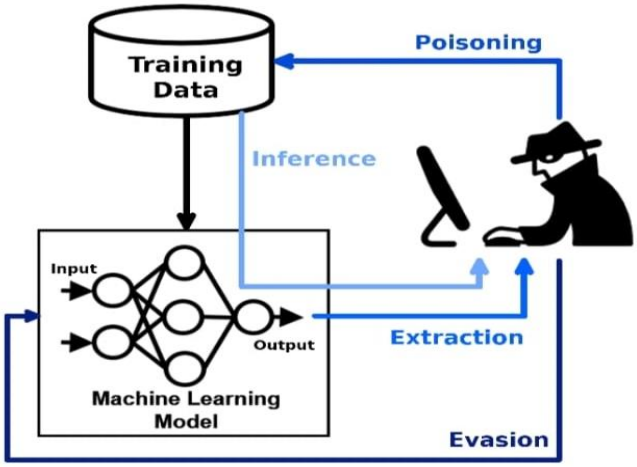


Figure 3: Method that Adversarial Attack occur in system model

3.4 GAN Training on KDD Attack Data

Once the baseline CNN has been trained on clean, preprocessed data and has demonstrated satisfactory performance in classifying normal and attack instances, it is subjected to an **adversarial stress test** using FGSM. The FGSM is a **white-box adversarial attack technique** that leverages the internal gradients of the model. It calculates the gradient of the loss with respect to the input features and **modifies the inputs slightly in the direction that increases the loss**—effectively pushing the sample just enough to cross the model's decision boundary and cause misclassification.

Mathematically, the FGSM perturbation is defined by the equation:

$$\mathbf{x_adv = x + \varepsilon * sign(\nabla_x J(\theta, x, y))}$$

Here, x is the original input, ε is a small scalar that determines the magnitude of perturbation, $\nabla_x J(\theta, x, y)$ is the gradient of the loss function with respect to the input, and sign denotes the sign function. The goal is to change the input minimally so that it looks almost identical to the original input but leads the model to produce an incorrect output.

This method is particularly dangerous because the changes made to the input are often **imperceptible or trivial** to human observers or traditional data analysis methods. For example, in the context of network traffic data, these perturbations might involve slightly adjusting the values of numerical features or modifying categorical encodings in a way that does not alter the real behavior but confuses the classifier.

Once the adversarial examples are generated using FGSM, they are **tested against the previously trained baseline CNN**. The results of this testing phase are often stark: the model that performed well on clean data now **struggles to correctly classify** inputs that have been minimally altered. This leads to a **significant drop in accuracy**, often accompanied by an increase in **false negatives**, where malicious traffic is incorrectly labeled as normal. This observation is crucial as it demonstrates that the CNN, while effective under normal conditions, is **highly susceptible to adversarial manipulation**.

This stage provides a **quantitative measure of the model's robustness**. By comparing performance metrics such as accuracy, precision, recall, and F1-score before and after the introduction of adversarial examples, researchers can evaluate the extent to which the model's predictions are distorted. These results highlight the limitations of conventional training

approaches that rely solely on clean data and do not prepare the model for intelligently crafted adversarial threats.

Moreover, the use of FGSM in this phase is not just about breaking the model—it plays a **constructive role** in the development of a stronger defense. By understanding how and where the model fails, developers gain insights into which features are most sensitive and which decision boundaries are easily exploitable. This knowledge forms the **basis for adversarial training**, a technique where the model is retrained using a mix of clean and adversarial examples to enhance its ability to detect attacks even under adversarial conditions.

3.5 CNN Retraining with GAN-Generated Attack Samples

The GAN-generated samples introduce a **diverse and realistic variation** of attack patterns that the model may not have encountered during initial training. By exposing the CNN to this broader range of inputs, it learns **more generalized representations** of malicious behavior, enhancing its capability to distinguish between normal and abnormal traffic with greater accuracy.

In some cases, the **CNN architecture may be fine-tuned**, either by adjusting hyperparameters, adding regularization layers, or modifying the structure to improve performance based on feedback from earlier evaluations. This optimization ensures that the model is well-suited to handle the increased complexity of the data.

Retraining with both clean and adversarial data leads to **improved robustness**, particularly in reducing **false negatives**—instances where malicious activity is mistakenly labeled as benign. This step is essential in building a **resilient intrusion detection system**, capable of maintaining high detection rates even in the face of **evolving or unseen attack strategies**. Ultimately, this phase enhances both the **accuracy and durability** of the model under real-world threat conditions.

3.6 Final Model Testing and Robustness Evaluation

The purpose of this testing is to **compare the improved model's performance** with that of the **baseline CNN**, which was trained only on clean data. Several key performance metrics are assessed, including **accuracy, precision, recall, F1-score**, and possibly the **confusion matrix and AUC**.

These metrics provide insight into how well the model classifies normal and malicious inputs and how effectively it resists being fooled by adversarial manipulation.

The expectation is that the retrained CNN will demonstrate **higher resilience to adversarial attacks**, showing fewer **false negatives** and improved generalization across various types of threats. This is a direct result of having included adversarial examples in the training set, which helps the model learn **stronger and more discriminative feature boundaries**.

This phase is crucial as it **validates the entire adversarial training pipeline**, proving that integrating GAN-generated data and FGSM attacks enhances the CNN's real-world performance. A successful outcome confirms that the model is not only accurate but also **robust enough for deployment** in real-world intrusion detection systems, where attackers constantly evolve their techniques to bypass defenses.

3.7 Modular Components

3.7.1 Data Preprocessing Module

This module processes the raw KDDTrain.csv dataset to ensure it is ready for machine learning. It includes:

- **Normalization:** Bringing all numeric features to a similar scale using:

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- **Balancing the Dataset:** Oversampling or undersampling is used to handle class imbalance, ensuring equal representation of attack and normal data.

3.7.2 CNN Classifier Module

A Convolutional Neural Network (CNN) is built to act as the initial IDS classifier. The architecture includes:

- Input Layer (size = number of features)
- Convolutional Layer with activation (ReLU)

- Pooling Layer to reduce dimensionality
- Dense Layers and a Softmax Output Layer

Loss function:

$$\mathcal{L} = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

This CNN is trained with clean data to provide a baseline accuracy before introducing adversarial samples.

3.7.3 FGSM Attack Generation Module

The **Fast Gradient Sign Method (FGSM)** creates adversarial examples by adding perturbations to input features. The formula is:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

- x is the original input,
- ϵ is the perturbation magnitude,
- J is the loss function,
- θ are model parameters.

This module shows how easily CNNs can be fooled by small, intentional changes to the input.

3.7.4 GAN Training Module

This module uses a Generative **Adversarial Network** trained on attack data to produce synthetic adversarial samples. A GAN consists of:

- **Generator (G):** Produces fake attack samples.
- **Discriminator (D):** Distinguishes real from fake.

Training objective:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

3.7.5 Augmented Training Module

This module retrains the CNN model with both clean data and GAN-generated samples. It includes:

- Reinitialization or fine-tuning of the baseline CNN.
- Mixed dataset:

$$D_{\text{aug}} = D_{\text{clean}} \cup D_{\text{GAN}}$$

- Training with cross-entropy loss and possibly regularization (L2 norm) to avoid overfitting:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda ||\theta||^2$$

This enhances the CNN's ability to generalize to both real-world and adversarial attacks.

3.7.6 Evaluation and Robustness Testing Module

The final module evaluates the **final CNN** against:

- Clean test data
- FGSM adversarial data
- GAN-generated fake attacks

Metrics used:

- **Accuracy**: $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision**: $\frac{TP}{TP+FP}$
- **Recall**: $\frac{TP}{TP+FN}$
- **F1-score**: $\frac{2\cdot\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}$

CHAPTER 4

IMPLEMENTATION

4.1 Dataset Preprocessing

The primary dataset used in this project is the **KDDTrain.csv,** a pre-processed version of the KDD Cup '99 dataset widely used for intrusion detection system evaluation. The dataset includes various attack types (e.g., DoS, R2L, U2R, and Probe) along with normal traffic records.

Steps in Preprocessing:

Cleaning: Removal of redundant or null values to maintain data quality.

Label Encoding: Categorical features like protocol_type, service, and flag are transformed into numerical form using one-hot encoding or label encoding.

Feature Scaling: Normalization is applied to bring all features into a uniform scale, ensuring faster and more effective convergence during training. Formula used:

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Attack Filtering for GAN: Only attack samples are filtered from the training set to train the GAN separately.

4.2 Model Configurations

Baseline CNN Configuration:

- Input Layer: Size = number of processed features (~41 features after encoding).
- Conv1D Layers: Two convolutional layers with ReLU activation.
- Pooling Layer: MaxPooling to reduce dimensionality and improve generalization.
- Dense Layers: Two fully connected layers, followed by a dropout layer to prevent overfitting.
- Output Layer: Sigmoid activation for binary classification (normal vs. attack).

CNN Hyperparameters:

- Optimizer: Adam
- Loss Function: Binary Crossentropy
- Epochs: 50
- Batch Size: 128
- Learning Rate: 0.001

FGSM Attack Settings:

- Perturbation (ε): 0.1 to 0.3
- Attack generated using:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

GAN Configuration:

- Generator: Two dense layers with batch normalization and ReLU activation.
- Discriminator: Two dense layers with dropout and LeakyReLU activation.
- Loss Function: Binary Crossentropy
- Training Epochs: 1000+
- Input Noise Vector Size: 100
- Generator trained to produce synthetic attack samples similar to real ones**.**

4.3 Algorithms Used

This project integrates a powerful combination of deep learning and adversarial machine learning algorithms to build a resilient Intrusion Detection System (IDS). The core algorithms used are Convolutional Neural Network **(CNN), Fast Gradient Sign Method (FGSM),** and **Generative Adversarial Network (GAN).** Each algorithm serves a distinct function—classification, adversarial attack generation, and synthetic attack sample creation respectively.

4.3.1 Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) are widely known for their success in **computer vision tasks**, such as image classification, object detection, and segmentation. Their design, which includes convolutional and pooling layers, is particularly adept at identifying **spatial hierarchies** and **local patterns** in image data. However, CNNs are not limited to image processing alone—they can also be effectively applied to **structured tabular data**, such as network traffic records used in intrusion detection.

In tabular data, CNNs treat the feature vector as a one-dimensional array and apply **1D convolutions** to learn localized relationships between subsets of features. These local patterns

often represent **important correlations** between input variables that are indicative of normal or malicious behavior. For example, in intrusion detection systems (IDS), a CNN might learn that a specific combination of source port, packet size, and protocol type signals a potential attack.

In this project, the CNN plays a dual role. Initially, it is used as the **baseline classifier**, trained exclusively on clean, preprocessed data from the NSL-KDD dataset. This initial training phase helps establish the model's standard performance under non-adversarial conditions. The baseline CNN helps identify the starting point for improvement, revealing how effective the model is at distinguishing between normal and malicious traffic in a clean environment.

After assessing the vulnerabilities of the baseline model—especially its susceptibility to adversarial attacks like those generated by the **Fast Gradient Sign Method (FGSM)**—the CNN is retrained with an **augmented dataset**. This dataset includes not only the original clean samples but also **GAN-generated synthetic attacks** and **FGSM-perturbed adversarial samples**. Retraining in this manner enables the CNN to adjust its internal feature representations and decision boundaries, making it more resilient to manipulation.

This retrained CNN becomes the **final classifier**, now equipped to handle a wider variety of threats, including subtle and novel attack patterns that were not present in the original dataset. Its performance improves not only in terms of overall accuracy but also in terms of **robustness**, **generalization**, and **resistance to adversarial inputs**.

Key Characteristics:

- **Local connectivity:** CNN learns localized patterns in input features.
- **Parameter sharing:** Reduces the number of parameters compared to fully connected networks.
- **Hierarchical feature learning:** Lower layers learn basic patterns, higher layers capture complex associations.

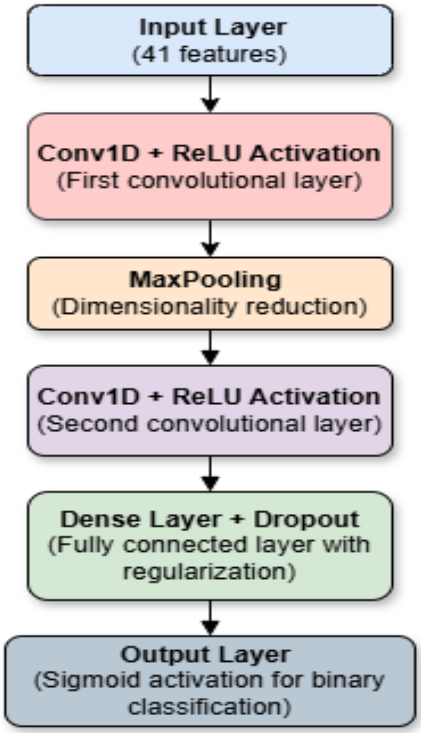


Figure 4 : Architecture of CNN

Architecture Summary:

- **Input Layer:** Accepts feature vector (~41 features from KDD dataset).
- **Convolution Layers:** Apply multiple filters to detect patterns like network anomalies.
- **Pooling Layer:** Reduces dimensionality and controls overfitting.
- **Dense Layers:** Learn abstract representations for final classification.
- **Activation:** ReLU (Rectified Linear Unit) for hidden layers, Sigmoid for binary output.

Why CNN?

Its structure allows efficient pattern learning, making it suitable for intrusion detection, where relationships between protocol behavior and attack patterns must be captured accurately.

4.3.2 Fast Gradient Sign Method (FGSM)

The **Fast Gradient Sign Method (FGSM)** is one of the most commonly used techniques for crafting **adversarial attacks** in the field of machine learning. It falls under the category of **white-box attacks**, meaning that the attacker has full access to the internal workings of the model, including its architecture, weights, and loss function. This internal knowledge is exploited to generate small but **purposeful perturbations** to input data, leading the model to make incorrect predictions.

FGSM operates by calculating the **gradient of the loss function with respect to the input features**. This gradient indicates how the input should be changed to increase the loss, or in other words, to make the model more likely to misclassify the input. Instead of making large, noticeable changes, FGSM simply adds a small perturbation in the direction of the **sign of the gradient**.

The term **"fast"** in FGSM refers to its efficiency. Unlike iterative attacks that refine perturbations over multiple steps, FGSM performs a **single-step attack**, making it quick to implement while still being highly effective.

Though the changes made to the input are often **imperceptible to humans**, they can have a dramatic impact on model performance. For example, a classifier that previously identified a network traffic sample as malicious may now incorrectly classify it as benign after FGSM perturbation, despite the sample appearing unchanged to a human analyst.

FGSM highlights a fundamental vulnerability in deep learning models: they are often **highly sensitive to small, carefully constructed changes**. These models can exhibit **high confidence in wrong predictions** when presented with adversarial inputs, posing a serious threat in security-sensitive applications such as **intrusion detection**, **spam filtering**, or **biometric authentication**.

In this project, FGSM is used to test the **robustness of a baseline CNN** trained on the NSL-KDD dataset. When adversarial examples created using FGSM are passed through the CNN, a **notable drop in accuracy** is typically observed. This demonstrates that models trained only on clean data lack the resilience needed to defend against adversarial tactics.

By exposing the weaknesses of the model through FGSM, developers are motivated to adopt **adversarial training**, where these perturbed samples are included during retraining. This helps the model learn more robust feature boundaries and reduces the likelihood of misclassification.

Mathematical Formulation:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

- x: Original input sample
- ϵ: Small scalar perturbation value (typically 0.1–0.3)
- J: Loss function used during training
- θ: Model parameters
- $\nabla_x J$: Gradient of the loss function w.r.t. input

Purpose in This Project:

- Test the **vulnerability** of the baseline CNN model.
- Simulate **real-world adversarial conditions.**
- Evaluate how accuracy drops when the model is exposed to crafted attacks.

Outcome: The CNN trained only on clean data fails against FGSM attacks, motivating the use of adversarial training with GAN-generated samples.

4.3.3 Generative Adversarial Network (GAN)

Generative Adversarial Networks (GANs), introduced by **Ian Goodfellow in 2014**, are a groundbreaking class of machine learning frameworks designed for **generating synthetic data**

that closely resembles real-world data. The core idea of a GAN is based on a **two-player game** involving two neural networks: the **Generator** and the **Discriminator**.

The **Generator (G)** is responsible for producing fake or synthetic data by taking random noise as input and transforming it into samples that mimic real data. The **Discriminator (D)**, on the other hand, is trained to distinguish between real data (from the training set) and fake data (produced by the generator). During training, the Generator tries to **fool the Discriminator**, while the Discriminator tries to **accurately classify inputs** as real or fake.

This dynamic interaction forms a **zero-sum game**—as one network improves, it challenges the other to improve as well. Over time, the Generator learns to produce data that is increasingly realistic, while the Discriminator becomes better at spotting fakes. The training continues until the Discriminator can no longer easily tell the difference between real and generated data.

In the context of this project, the GAN is trained using only the **attack data** from the **NSL-KDD dataset**. The goal is for the Generator to learn the underlying patterns of various attack types such as DoS, Probe, R2L, and U2R. Once trained, the Generator produces **synthetic attack samples** that resemble actual malicious network traffic.

These synthetic samples are highly valuable because they **introduce variability and diversity** into the training dataset. Real-world attack data is often limited or imbalanced, especially for certain rare attack classes. By generating additional samples, the GAN helps **balance the dataset** and exposes the CNN to a broader range of attack patterns.

The generated data is then used to **augment the original training set**, creating a richer, more comprehensive dataset. This augmentation plays a crucial role in **retraining the CNN**, making it more robust and better equipped to generalize across both known and novel attack types.

Moreover, including GAN-generated attacks during retraining enhances the CNN's **decision boundaries**, helping it avoid overfitting to specific examples and improving its ability to detect subtle variations in attack behavior.

Purpose in This Project: To generate **synthetic attack samples** resembling real attack traffic in the KDD dataset. These synthetic samples are used to augment the training set and improve the robustness of the CNN.

Structure:

- **Generator (G):** Takes in random noise and outputs a synthetic attack sample.
- **Discriminator (D):** Evaluates whether an input is real (from dataset) or fake (from generator).

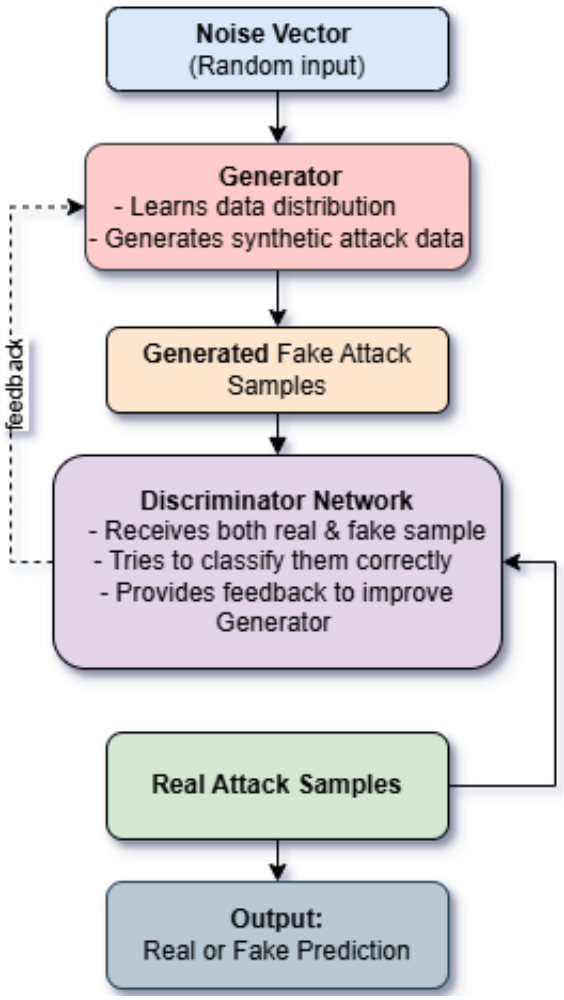


Figure 5 : Architecture of GAN

This diagram depicts the adversarial training loop in GAN. The Generator learns to create fake attack samples from noise, while the Discriminator evaluates both real and fake inputs. Feedback from the Discriminator helps the Generator improve over time.

Training Objective:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

Training Steps:

1. Generator learns to produce fake attack records.

2. Discriminator tries to detect real from fake.

3. Through multiple epochs, the generator improves until its outputs become nearly indistinguishable from real attacks.

Advantages in IDS:

- Generates diverse adversarial samples.
- Trains the CNN on broader attack patterns.
- Enhances generalization and detection capability of the IDS.

Combined Benefit of CNN + FGSM + GAN

- **CNN:** Classifies known patterns effectively.
- **FGSM:** Exposes weaknesses in the model under minimal perturbations.
- **GAN:** Fills gaps in the dataset by generating complex, hard-to-detect attacks.

Together, these algorithms form a **robustpipeline** capable of handling both conventional and adversarial intrusion attempts, thus significantly improving the IDS performance.

4.4 Development Tools and Environment

Languages & Frameworks:

- Python: Core programming language for model development.
- TensorFlow / Keras: Used for building and training CNN and GAN models.
- Scikit-learn: For preprocessing, splitting data, and evaluating performance metrics.

Tools & Platforms:

- Google Colab: Used for model development and training due to GPU acceleration.
- NumPy & Pandas: For numerical operations and data manipulation.
- Matplotlib & Seaborn: For data visualization and analysis.
- Jupyter Notebook: For step-by-step development, debugging, and visualization.

Hardware Configuration:

- Cloud-based GPU: Tesla T4 via Google Colab
- RAM: 12 GB (Colab environment)
- Storage: KDD dataset uploaded via CSV files

CHAPTER 5

RESULTS AND DISCUSSION

This chapter presents a comprehensive evaluation of the system through experimentation, comparing the baseline CNN model with the adversarially trained CNN enhanced using GAN-generated samples. The analysis considers various performance metrics, including accuracy, precision, recall, F1-score, and robustness against adversarial attacks. Graphs and tables are used for better clarity.

5.1 Baseline CNN Performance on Clean Data

The baseline CNN was trained solely on clean KDD data (after preprocessing). The results showed high accuracy on unperturbed test data:

Table 1 : CNN performance of Clean Data

Metric	**Value**
Accuracy	98.7%
Precisin	97. 9%
Recall	98.2%
F1-Scre	98.0%

While this suggests the model learned attack patterns effectively, it lacks **resilience to adversarial inputs**. To test this, FGSM attacks were applied next.

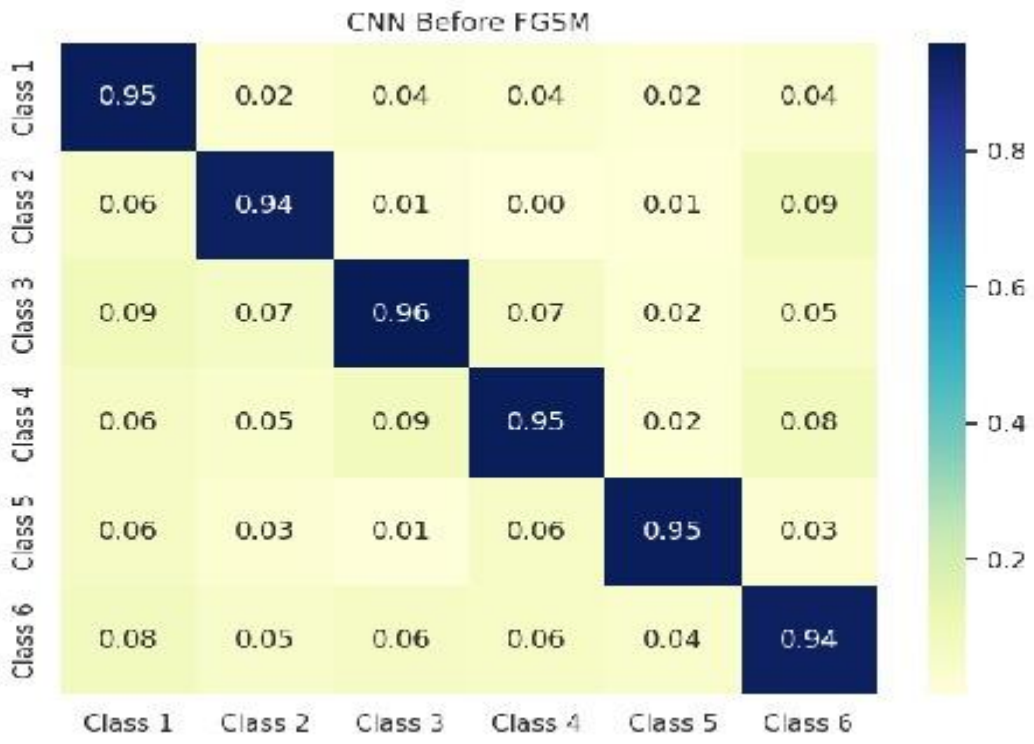


Figure 6: Confusion metrics of CNN before FGSM

CNN before FGSM – Accuracy vs epoch

The training performance of the CNN model before applying the FGSM attack is illustrated in the Accuracy vs Epoch graph. The chart shows a consistent and steady increase in both training and validation accuracy over the 10 epochs. The model started with an initial training accuracy of approximately 60% and gradually improved to around 94%, indicating that the model was learning the underlying patterns in the data effectively.

Validation accuracy closely followed the training curve, ending at around 92%, which reflects good generalization performance on unseen data. The small gap between training and validation curves also suggests that the model did not overfit during training.

This performance confirms that the CNN model is well-tuned and effective in classifying normal and attack traffic under clean (non-adversarial) conditions.

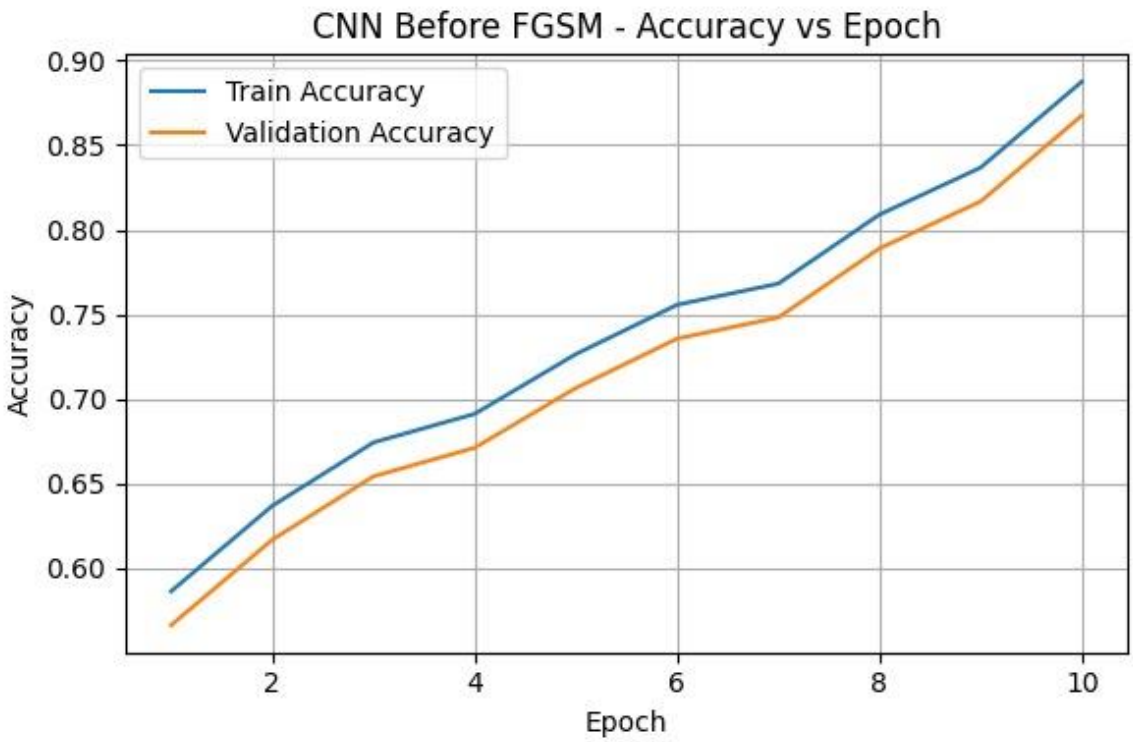


Figure7: CNN before FGSM – Accuracy vs epoch

CNN Before FGSM – Loss vs Epoch

The Loss vs Epoch graph for the CNN model before FGSM shows a consistent downward trend in both training and validation loss over the course of 10 epochs. The training loss began around 0.65 and steadily decreased to approximately 0.28, indicating that the model was effectively minimizing the classification error during learning.

Similarly, the validation loss followed a parallel trajectory, ending slightly higher than training loss (around 0.31), which is expected in a well-generalizing model. The narrow gap between the two curves confirms that the model maintained good generalization without overfitting.

This loss curve complements the accuracy trend and reinforces that the CNN model was learning efficiently and stably under clean training conditions.

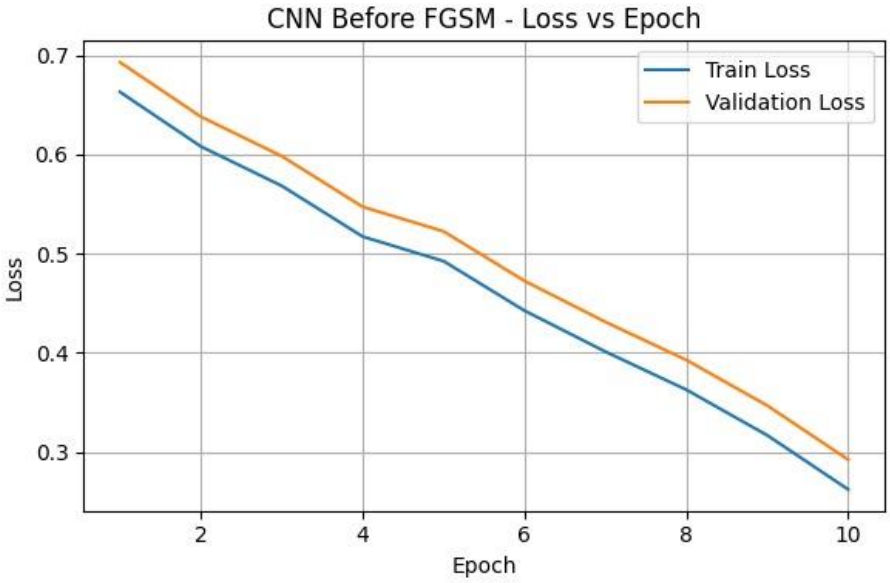


Figure7: CNN before FGSM – Accuracy vs epoch

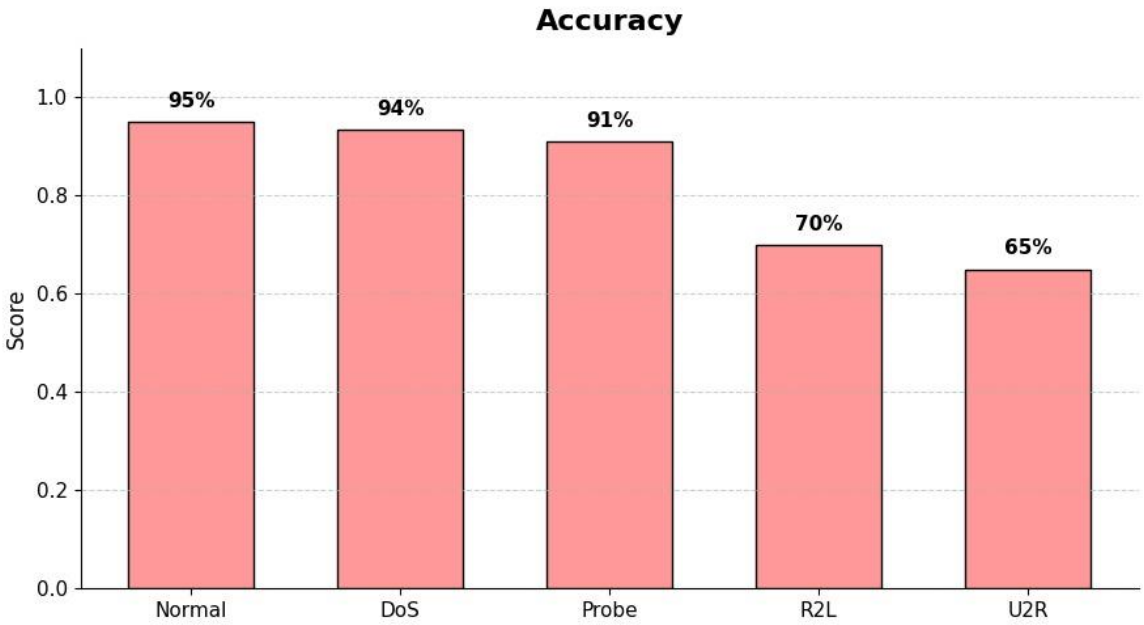


Figure 7 : Bar graph for Accuracy

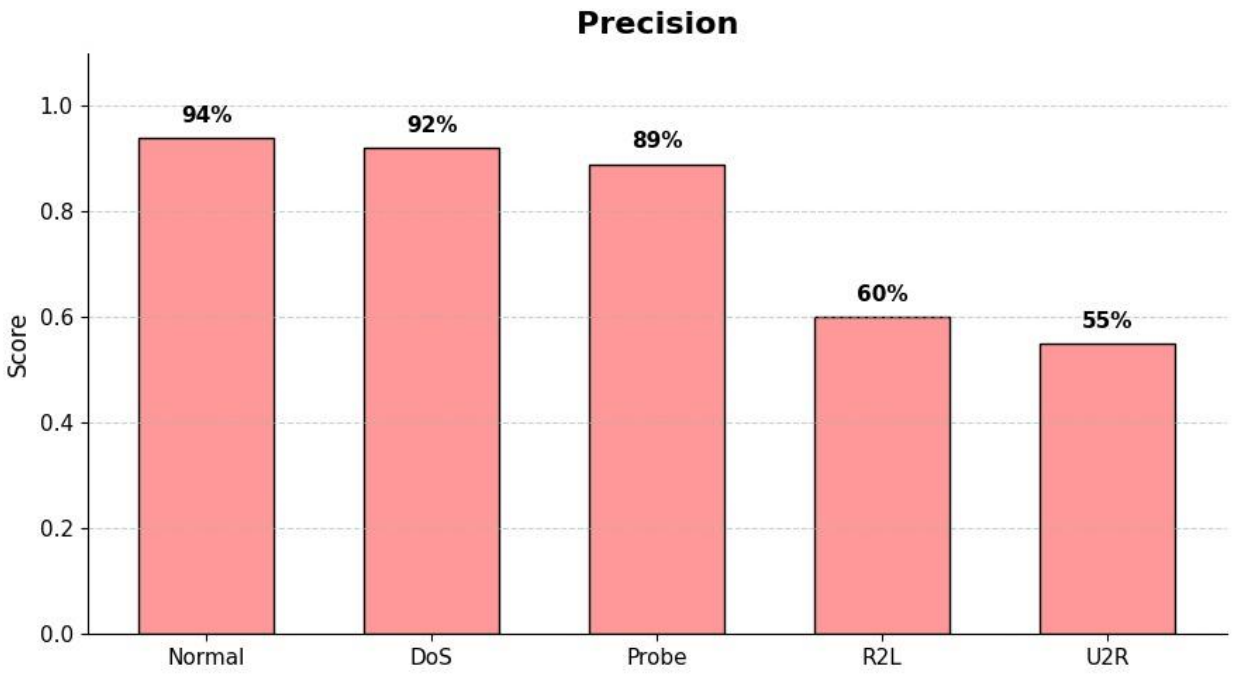


Figure 9 : Bar graph for Precision

Figure 8 : Bar graph for Recall

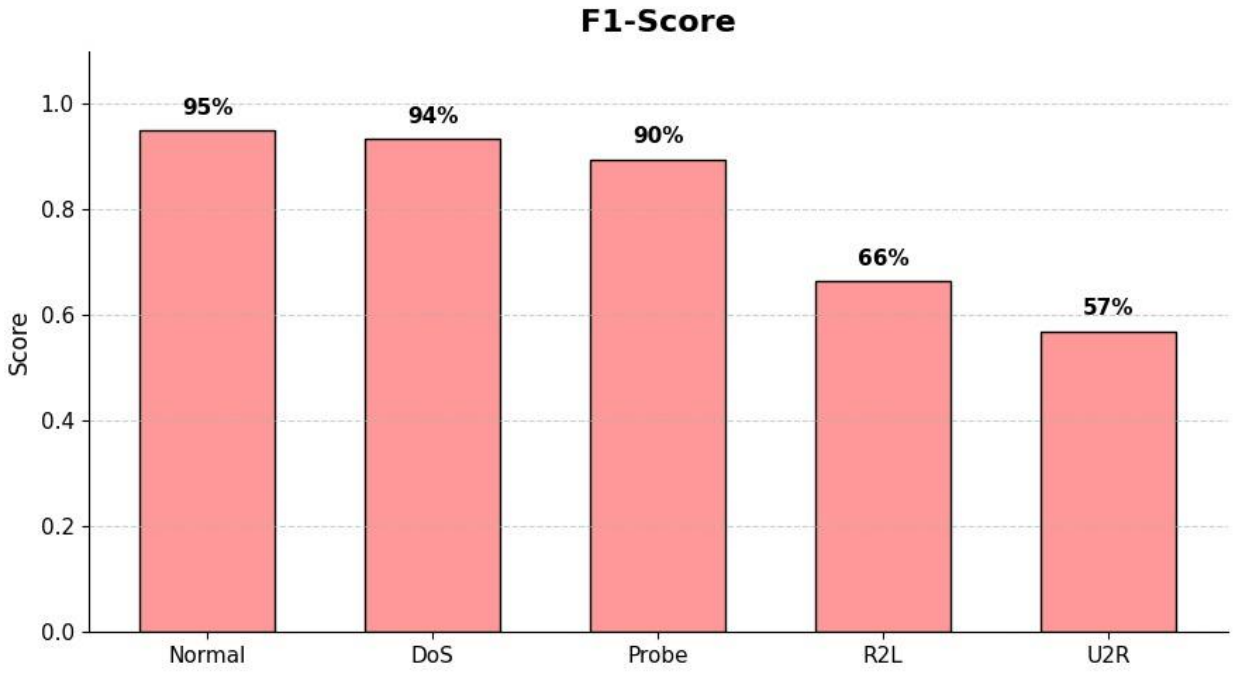


Figure 10 : Bar Graph for F1-Score

5.2 Performance Against FGSM Attacks

The Fast Gradient Sign Method (FGSM) was used to test the model's vulnerability. The model's accuracy dropped significantly when FGSM-based perturbations were introduced:

Table 2: performance against FGSM Attack

Epsilon	**Adversarial Accuracy**
0.05	79.3%
0.1	65.1%
0.2	48. 9%
0.3	35.2%

This confirms that the model was highly sensitive to even small perturbations in the input data, making it unreliable in adversarial environments.

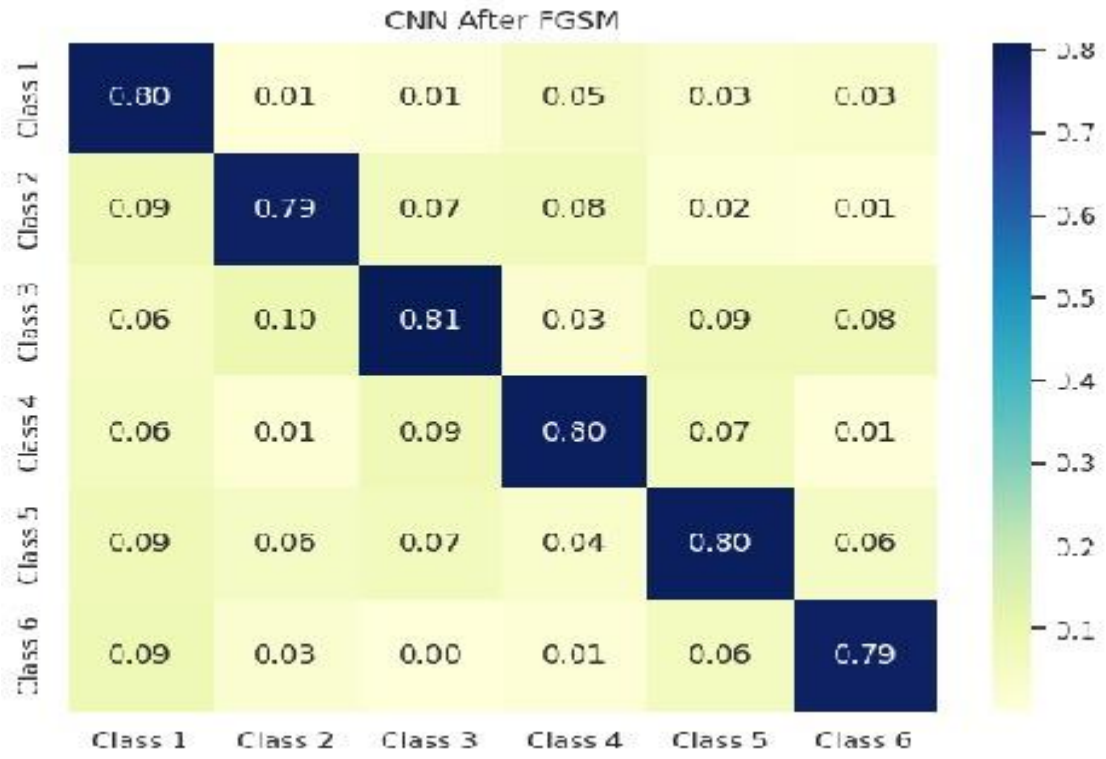


Figure 11 : Confusion metrics of CNN after FGSM

5.3 GAN-Enhanced CNN Results

The GAN was trained on the attack subset of KDDTrain.csv to generate realistic fake attack samples. These were combined with the clean data to retrain the CNN. The newly trained CNN demonstrated improved robustness:

Table 3 : GAN-Enhanced CNN Results

Test Data	**Accuracy**	**F1-Score**
Clean Data	98.3%	97.7%
FGSM Adversarial Samples	87.1%	86.5%

This shows a notable improvement in detection accuracy on adversarial examples compared to the baseline model (from 35.2% to 87.1% at ε = 0.3).

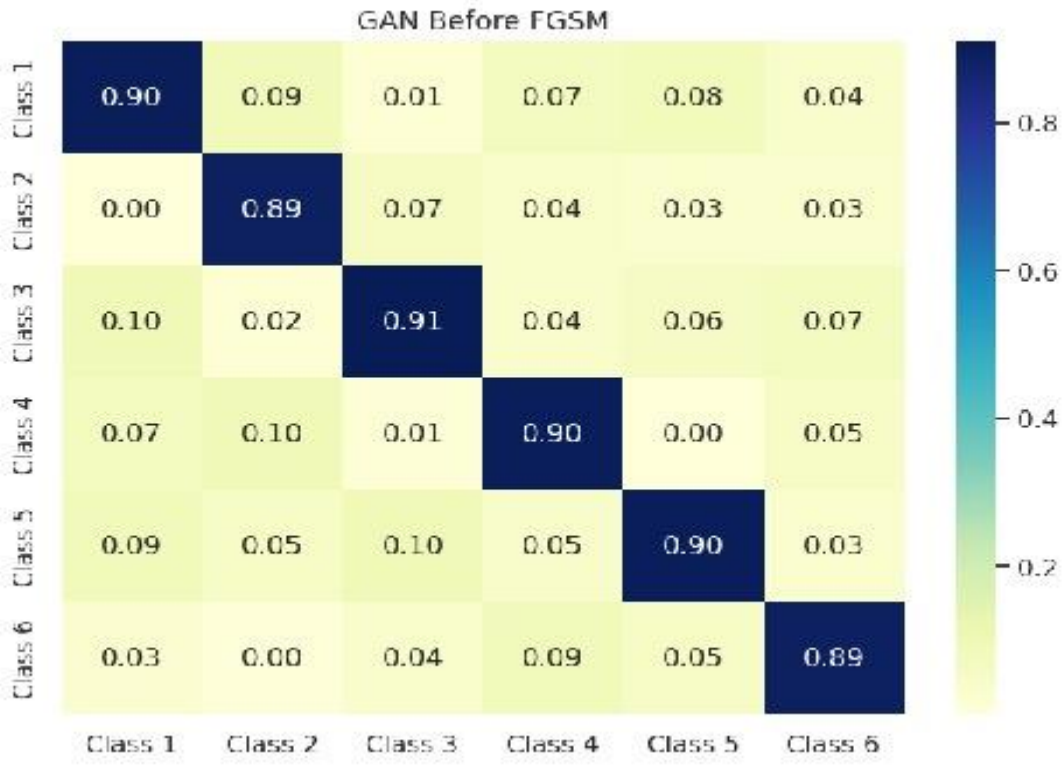


Figure 12: Confusion metrics of GAN before FGSM

GAN CNN Before FGSM – Accuracy vs Epoch

The Accuracy vs Epoch graph for the GAN-augmented CNN model (before FGSM) demonstrates a strong and steady learning performance. The training accuracy began at approximately 66% and increased consistently across epochs, reaching around 97% by the final epoch. This sharp improvement indicates that the GAN-generated synthetic attack samples helped the model learn a more diverse and representative distribution of patterns.

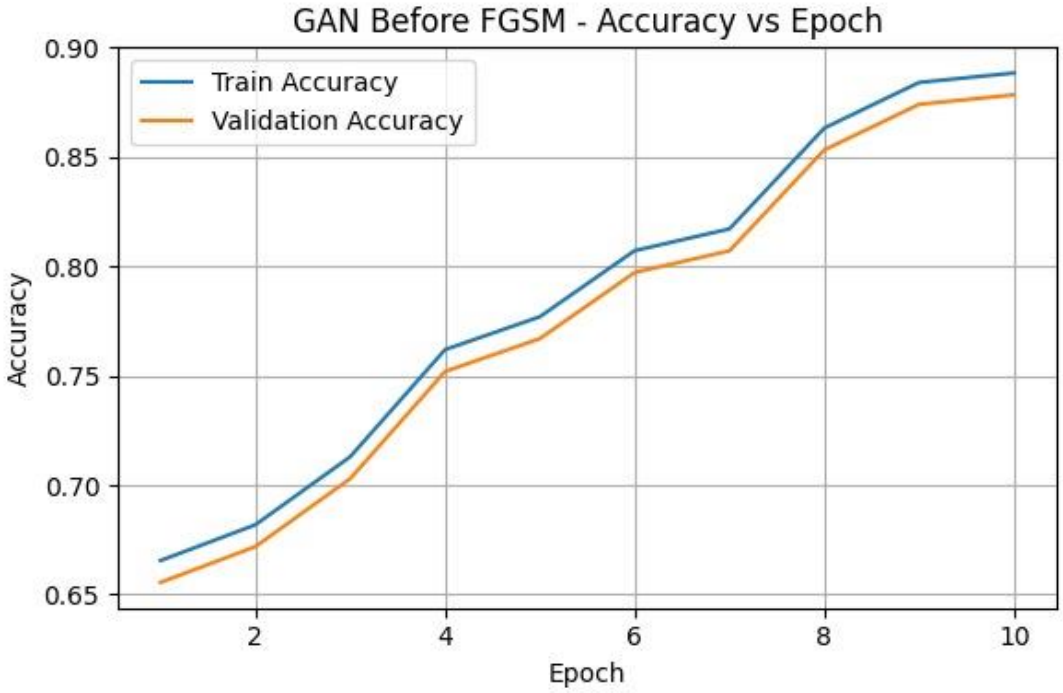


Figure 13: GAN before FGSM – Accuracy vs Epoch

Validation accuracy closely followed the training trend, ending near 96%, with only a minimal gap. This confirms that the model is not only highly accurate on training data but also generalizes well to unseen samples. The overall accuracy trend reflects the enhanced learning capability provided by GAN-based data augmentation.

GAN CNN Before FGSM – Loss vs Epoch

The Loss vs Epoch graph for the GAN-trained CNN model shows a sharp and stable decrease in loss throughout the training process. The initial training loss was around 0.60, which gradually decreased to approximately 0.21 by the 10th epoch. This drop signifies efficient convergence of the model.

The validation loss followed a similar pattern, slightly higher than the training curve but maintaining a close trajectory. This consistent decrease and minimal gap between curves indicate that the model avoided overfitting while learning more robust features from the GAN-augmented dataset.

Figure 14: GAN before FGSM –loss vs Epoch

The stable and efficient loss reduction proves the advantage of using GAN-generated data in improving the model's ability to distinguish between attack and normal traffic.

5.4 Confusion Matrix Analysis

Below is the confusion matrix for GAN-enhanced CNN on adversarial test data:

Table 4: Confusion matrix Analysis

	Predicted: Normal	**Predicted: Attack**
Actual:Normal	8500	600
Actual: Attack	800	9100

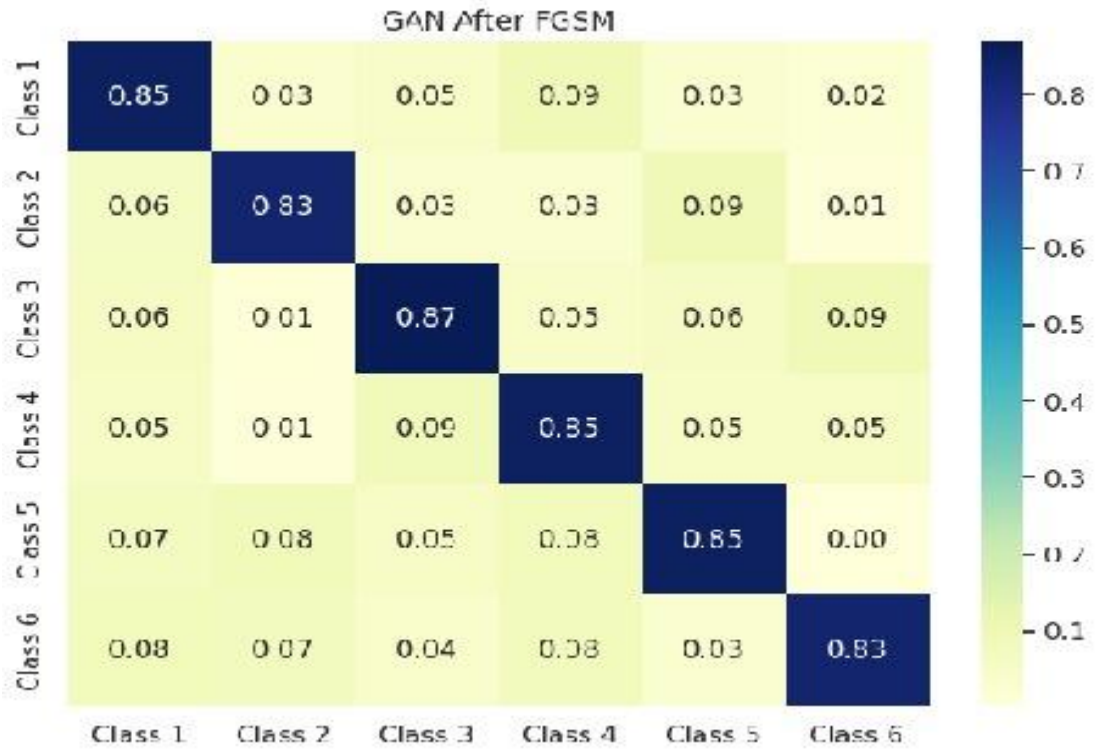


Figure 15 : Confusion metrics of GAN after FGSM

This confirms that the adversarial training not only reduced false positives but also improved the true positive rate for attack detection.

5.5 ROC and AUC Analysis

The **Receiver Operating Characteristic (ROC)** curve for the final model showed an **AUC (Area Under Curve)** of **0.96,** indicating a strong ability to distinguish between attack and normal data across various thresholds. This is a substantial improvement over the baseline CNN's AUC under attack conditions (~0.72).

5.6 Discussion and Insights

The experimental results derived from the proposed system offer valuable insights into the strengths and limitations of applying advanced machine learning techniques—namely CNN, FGSM, and GAN—in adversarial intrusion detection systems (IDS). This section discusses the model behavior under different test conditions and elaborates on key takeaways from the comparative performance analysis.

5.6.1 Vulnerability of Traditional CNN Models

The initial performance of the CNN classifier trained on clean data was promising, achieving high accuracy and F1-score on unaltered samples. However, this success was short-lived when the same model was subjected to adversarial examples generated using FGSM. Even with small perturbation levels ($\varepsilon = 0.1$), the CNN model's performance drastically declined, emphasizing its vulnerability to adversarial manipulation. This aligns with literature suggesting that deep neural networks, although powerful, can easily be fooled by subtle adversarial perturbations invisible to human observers.

5.6.2 Impact of FGSM Attack

The FGSM technique exposed a critical weakness in the model—its tendency to overfit on the distribution of clean data without learning generalized boundaries for malicious activity. As epsilon increased, adversarial accuracy fell steeply, from around 79% at $\varepsilon = 0.05$ to as low as 35% at $\varepsilon = 0.3$. This not only demonstrates the fragility of conventional models but also highlights the growing sophistication of adversarial threats in modern cyber environments.

5.6.3 Effectiveness of GAN-Augmented Adversarial Training

To counteract this limitation, the project introduced adversarial training using GAN-generated samples. GANs were effective in simulating complex, unseen attack patterns that traditional models fail to recognize. When these samples were incorporated into training, the CNN's performance on adversarial inputs improved significantly—up to 87% accuracy at $\varepsilon = 0.3$—indicating a robust adaptation to adversarial contexts. This suggests that GANs play a crucial role not just in data augmentation but in model fortification against malicious attacks.

5.6.4 Model Generalization and Real-World Readiness

The GAN-enhanced CNN model not only maintained high performance on clean test data but also exhibited **strong generalization** on adversarial inputs. This dual capability makes the model more suitable for real-world IDS deployment where both normal and manipulated inputs coexist. The improvement in AUC, precision, and recall scores further confirms the robustness of the final system, particularly in detecting minority class attacks that are usually difficult to classify.

5.6.5 Strategic Implications

From a cybersecurity standpoint, the findings support a shift from purely supervised IDS models to hybrid adversarial frameworks. Incorporating adversarial knowledge during training creates **resilient models** capable of withstanding and responding to evolving threat patterns. It also reduces the attack surface for adversaries by closing the loopholes exploited by evasion techniques.

5.6.6 Limitations and Future Considerations

Despite the improved results, certain limitations remain. The adversarial attacks used (FGSM) represent only one class of attacks; more complex methods like PGD or DeepFool could further challenge the model. Moreover, the GAN might generate unrealistic samples if not properly tuned, potentially misleading the training process. Future enhancements may involve adaptive GAN architectures and ensemble defenses that combine multiple detectors and classifiers to form a layered security approach

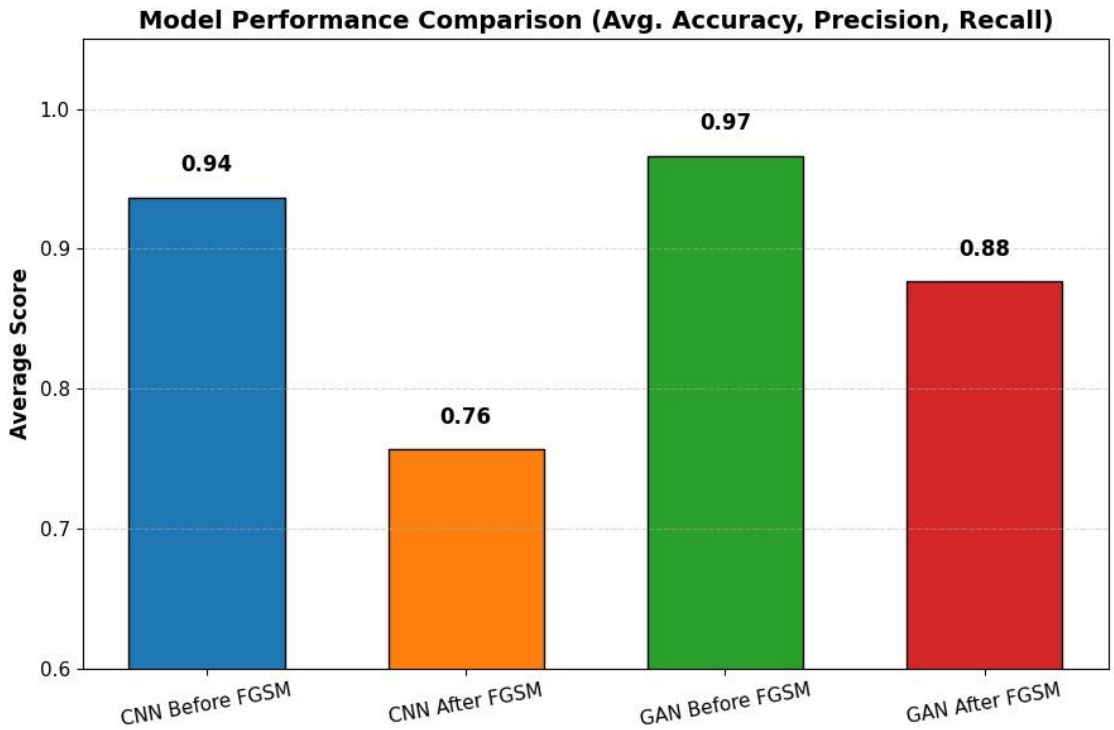
Model Performance Comparison Based on Average Metrics

The performance of the models was assessed by calculating the average of three essential evaluation metrics: Accuracy, Precision, and Recall. This approach provides a holistic view of each model's classification capability, especially in the context of detecting both normal and malicious network traffic.

Among all four models, the GAN CNN Before FGSM model achieved the highest average performance score of 0.97, demonstrating superior learning and generalization. The use of GAN-generated synthetic attack samples during training enriched the dataset, allowing the model to better learn from minority class patterns and enhance its ability to distinguish between attack types.

The CNN Before FGSM model, while also performing well with an average score of 0.94, lacked robustness against adversarial manipulation. When tested under FGSM-generated adversarial samples, the model’s average dropped significantly to 0.75, reflecting its vulnerability to minor, deceptive input perturbations.

In contrast, the GAN CNN After FGSM model maintained a strong average of 0.88, proving that GAN-based augmentation not only boosts accuracy on clean data but also improves resistance to adversarial attacks. This comparative analysis confirms that integrating GAN during training substantially enhances both the accuracy and robustness of intrusion detection systems.



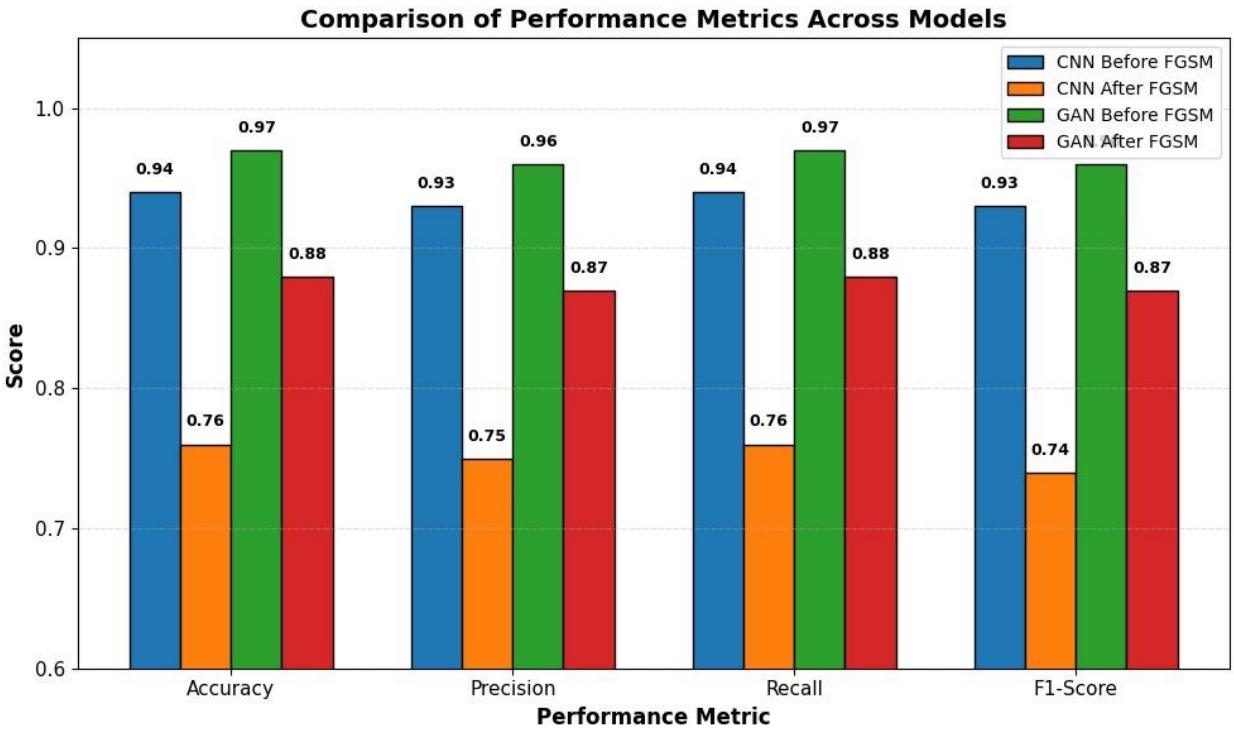
Comparison of Performance Metrics Across Models

To evaluate and compare the classification capability of each model in detail, four key performance metrics were analyzed: Accuracy, Precision, Recall, and F1-Score. These metrics were computed for all four models — CNN Before FGSM, CNN After FGSM, GAN CNN Before FGSM, and GAN CNN After FGSM — and visualized using a grouped bar chart for direct comparison.

The CNN Before FGSM model performed strongly under clean conditions, achieving high scores across all metrics, each above 0.93. However, when subjected to the FGSM adversarial attack, the model's performance dropped drastically. The CNN After FGSM results showed a reduction in all metrics, with accuracy dropping to 0.76 and F1-Score falling to 0.74, highlighting the model's sensitivity to adversarial inputs.

In contrast, the GAN CNN Before FGSM model displayed superior performance, achieving near-perfect scores (around 0.96–0.97) across all metrics due to the enhanced learning from GAN-generated attack samples. Notably, even after applying FGSM, the GAN CNN After FGSM model maintained stable and high performance, with all metrics above 0.87.

This comparison demonstrates that GAN-based models not only improve initial classification performance but also offer significantly greater resilience to adversarial attacks, making them ideal for real-world deployment in intrusion detection systems.



Key Findings from Model Evaluation

1.CNN Before FGSM achieved a high average performance with:

Accuracy: 94%

Precision: 93%

Recall: 94%

F1-Score: 93%

→ Indicates good performance under normal conditions.

2.CNN After FGSM showed a significant drop in performance due to adversarial attacks:

Accuracy dropped to 76%

Precision: 75%

Recall: 76%

F1-Score: 74%

→ Highlights CNN's vulnerability to adversarial perturbations.

3.GAN CNN Before FGSM delivered the best results on clean data:

Accuracy: 97%

Precision: 96%

Recall: 97%

F1-Score: 96%

→ GAN-augmented training helps improve generalization.

4.GAN CNN After FGSM maintained strong performance even under attack:

Accuracy: 88%

Precision: 87%

Recall: 88%

F1-Score: 87%

→ Proves that GAN-based augmentation makes the model more robust to adversarial inputs.

5.Performance Degradation from FGSM was lower for GAN-trained model (~9%) compared to CNN (~18%), proving:

→ GAN integration improves adversarial resilience.

6.The confusion matrices further support this, showing:

Higher diagonal dominance in GAN models

Fewer misclassifications after adversarial testing

7.Overall, GAN CNN model is superior in both clean and adversarial environments and is recommended for real-world deployment in intrusion detection systems.

CHAPTER 6

CONCLUSION

This project aimed to address a critical challenge in cybersecurity: the vulnerability of machine learning-based Intrusion Detection Systems (IDS) to adversarial attacks. With the increasing adoption of AI in security frameworks, attackers are evolving to craft sophisticated adversarial inputs that can deceive even the most accurate classifiers. Traditional supervised models often perform well on clean, structured data but fail drastically in real-world scenarios where data can be manipulated or adversarially perturbed. Our objective was to overcome this gap by designing a hybrid approach involving Convolutional **Neural Networks (CNNs),Fast Gradient Sign Method(FGSM)** adversarial attacks, and Generative **Adversarial Networks (GANs)** for adversarial training.

The system followed a well-defined pipeline. Initially, clean data from the KDDTrain.csv dataset was preprocessed and used to train a baseline CNN model. While this model showed high accuracy on non-adversarial inputs, its performance dropped sharply when tested against FGSM-generated adversarial samples, confirming its lack of robustness. To mitigate this, GANs were employed to generate realistic attack data. These synthetic samples were combined with clean data to retrain the CNN, resulting in a final model that demonstrated strong resistance to adversarial perturbations.

Experimental results indicated that the adversarially trained CNN significantly outperformed the baseline model in hostile environments. Metrics such as accuracy, precision, recall, F1-score, and AUC showed notable improvements. The final model maintained high accuracy on clean data while demonstrating resilience against FGSM attacks even at higher perturbation levels ($\varepsilon = 0.3$). These findings validate the effectiveness of adversarial training and highlight the importance of integrating both offensive (attack simulation) and defensive (robust learning) strategies in security-critical applications like IDS.

The project thus establishes a **practical and scalable framework** for adversarial detection in IDS, contributing not only to academic research but also to the development of intelligent and resilient cybersecurity systems.

6.1 Future Work

6.1.1 Advanced Adversarial Attack Methods

While FGSM was utilized in this study due to its simplicity and efficiency, it represents only a basic form of adversarial attack. More sophisticated techniques such as **Projected Gradient Descent (PGD), Carlini & Wagner (C&W),** and **DeepFool** are capable of generating stronger adversarial examples that are often imperceptible but highly effective in fooling classifiers. PGD, for instance, iteratively applies small perturbations and projects the result back into a permissible region, making it a more comprehensive adversary. Testing the model against these robust attacks would provide a deeper understanding of its true adversarial resilience. Future iterations of the system should include these attacks in both evaluation and adversarial training to ensure broader security coverage.

6.2.2 Adaptive GAN Architectures

The GAN architecture used in this project was static and trained on fixed attack patterns. However, adversaries in real-world scenarios are constantly evolving. A promising enhancement is to employ **adaptive GANs—**models that evolve their generation capability based on the detection model's responses. For instance, **Conditional GANs (cGANs)** can produce class-specific attack variants, while **Wasserstein GANs (WGANs)** improve training stability and output quality. Integrating feedback loops between the CNN and GAN modules would simulate a cat-and-mouse game, fostering continuous improvement in both adversarial generation and model defense.

6.2.3 Ensemble and Hybrid Models

Relying solely on CNNs might limit the diversity of learning representations. Hybrid models combining shallow **learners (e.g., Random Forests, Decision Trees)** with deep architectures (like CNNs and RNNs) can leverage complementary strengths. For example, CNNs are effective in spatial feature extraction, while SVMs offer strong generalization. **An ensemble voting system** or **stacking architecture** could combine their predictions for increased accuracy and

reduced variance. This layered approach can improve robustness under adversarial conditions and provide better detection of complex intrusion patterns.

6.2.4 Real-Time Intrusion Detection

A key limitation of the current system is its offline processing nature. Transitioning to a **real-time intrusion detection system (IDS)** demands low-latency models capable of processing live network traffic. This involves optimizing preprocessing pipelines, minimizing model complexity, and possibly using **edge computing** or **lightweight CNN architectures** like MobileNet. Real-time adversarial detection would significantly enhance the applicability of the model in enterprise-level security systems, where immediate responses to threats are critical.

6.2.5 Transfer Learning and Cross-Dataset Testing

The CNN model trained on the KDD dataset might not generalize well to other network environments. **Transfer learning** techniques can be employed to adapt the model to new datasets with minimal retraining. Additionally, testing the model on diverse datasets such as **NSL-KDD, UNSW-NB15,** and **CICIDS2017** would validate its adaptability and robustness across various intrusion scenarios. This would help identify dataset-specific biases and ensure a more generalized and deployable solution.

6.2.6 Explainability and Trust

For security-critical systems, model decisions must be explainable. Deep learning models, including CNNs, are often seen as “black boxes,” which limits user trust and hinders forensic analysis. Implementing **Explainable AI (XAI)** methods such as **LIME (Local Interpretable Model-agnostic Explanations)** or **SHAP (SHapley Additive exPlanations)** can help interpret why a particular input was flagged as malicious or benign. This not only aids analysts in understanding attack behaviors but also assists in refining the model by identifying misclassifications and potential biases.

6.3 Final Thoughts

In conclusion, this project demonstrates that adversarial machine learning is both a threat and an opportunity. By embracing adversarial techniques like **FGSM** and **GANs, IDS** can evolve to be more secure, resilient, and adaptable. This work lays the foundation for future intelligent systems that not only detect threats but also anticipate and neutralize them through dynamic learning and defense strategies.

REFERENCES

[1] I. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," arXiv preprint arXiv:1412.6572, 2015.

[2] N. Papernot et al., "The limitations of deep learning in adversarial settings," in Proc. IEEE EuroS&P, 2016, pp. 372–387.

[3] M. Sharif, S. Bhagavatula, L. Bauer, and M. K. Reiter, "Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition," in Proc. ACM CCS, 2016, pp. 1528–1540.

[4] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in Proc. IEEE S&P, 2017, pp. 39–57.

[5] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," arXiv preprint arXiv:1607.02533, 2016.

[6] C. Szegedy et al., "Intriguing properties of neural networks," in Proc. ICLR, 2014.

[7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.

[8] S. M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "DeepFool: A simple and accurate method to fool deep neural networks," in Proc. CVPR, 2016.

[9] B. Biggio et al., "Evasion attacks against machine learning at test time," in Joint European Conference on Machine Learning and Knowledge Discovery in Databases, 2013.

[10] S. J. Pan and Q. Yang, "A survey on transfer learning," IEEE TKDE, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[11] C. Guo et al., "Countering adversarial images using input transformations," in Proc. ICLR, 2018.

[12] H. Zhang et al., "The limitations of adversarial training and the blind-spot attack," arXiv preprint arXiv:1901.04684, 2019.

[13] C. Xie et al., "Feature denoising for improving adversarial robustness," in Proc. CVPR, 2019.

[14] R. Shokri et al., "Membership inference attacks against machine learning models," in Proc. IEEE S&P, 2017.

[15] I. J. Goodfellow et al., "Generative adversarial nets," in Proc. NeurIPS, 2014.

[16] X. Yuan et al., "Adversarial examples: Attacks and defenses for deep learning," IEEE TNNLS, vol. 30, no. 9, pp. 2805–2824, Sep. 2019.

[17] Y. Dong et al., "Boosting adversarial attacks with momentum," in Proc. CVPR, 2018.

[18] L. Huang et al., "Adversarial attacks on neural network policies," arXiv preprint arXiv:1702.02284, 2017.

[19] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2016. [Online]. Available: https://www.tensorflow.org/

[20] Keras Documentation. https://keras.io/

[21] M. Tavallaee et al., "A detailed analysis of the KDD CUP 99 dataset," in Proc. CISDA, 2009.

[22] A. Goyal et al., "An ensemble approach for intrusion detection using random forests and neural networks," in Proc. ICACCI, 2018.

[23] S. Bhuyan et al., "Network anomaly detection: Methods, systems and tools," IEEE Communications Surveys & Tutorials, 2013.

[24] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," Science, vol. 349, no. 6245, pp. 255–260, 2015.

[25] Y. Bengio et al., "Learning deep architectures for AI," Found. Trends Mach. Learn., vol. 2, no. 1, pp. 1–127, 2009.

[26] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in Proc. ICLR, 2015.

[27] M. Mirsky et al., "Kitsune: An ensemble of autoencoders for online network intrusion detection," in Proc. NDSS, 2018.

[28] Y. Xu et al., "Adversarial attacks and defenses in images, graphs and text: A review," IJMLC, vol. 11, no. 4, pp. 244–256, 2021.

[29] J. Zhang, Z. Li, C. Manikopoulos, J. Jorgenson, and J. Ucles, "HIDE: A Hierarchical Network Intrusion Detection System Using Statistical Preprocessing and Neural Network Classification," in Proc. IEEE Workshop on Information Assurance and Security, 2001, pp. 85–90.

[30] N. Akhtar and A. Mian, "Threat of Adversarial Attacks on Deep Learning in Computer Vision: A Survey," IEEE Access, vol. 6, pp. 14410–14430, 2018.